۱۶ آبان ۱۴۰۵ | نسخه ۱.۱ راهنما

راهنمای اتصال شریک‌ها به فان‌لندز

این سند فان‌لندزه برای تیم فنی هر شریکی که می‌خواد بازی‌های فان‌لندز رو داخل سرویس خودش نشون بده. تیم‌های داخلی خودمون هم که روی ماژول شریک یا کلاینت بازی کار می‌کنن همین رو مبنا می‌ذارن. پایه این متن قرارداد یکپارچه‌سازی فان‌لندز نسخه ۲.۰ هست. اینجا همون قرارداد رو به زبان اجرایی نوشتیم و هر جا چیزی باز مونده بود، تصمیم خودمون رو گذاشتیم. توی بخش ۱۰ هم نمونه کد NestJS برای همه کارهای سمت شریک اومده. اگه جایی این متن با قرارداد نخونه، قرارداد ملاکه و لطفا به ما خبر بدید.

اصل اعتماد

هر طرف با کلید خصوصی خودش امضا می‌کنه و کلید عمومیش رو روی یک نشانی JWKS منتشر می‌کنه. هیچ کلید خصوصی یا سکرت مشترکی بین ما و شریک جابه‌جا نمیشه.

هر پوسته یک شریک

هر پوسته یک شریک جداست با شناسه، کلید، داده و پنل خودش. هیچ شریکی داده شریک دیگه رو نمی‌بینه.

دو محیط جدا

sandbox و production کاملا از هم جدا هستن: کلید، issuer، client_id، audience، داده، وب‌هوک و پنل. توکن یک محیط توی محیط دیگه رد میشه و یک `sub` یکسان توی این دو محیط دو بازیکن جدا در فان‌لندزه.

نمونه‌ها

دامنه‌هایی مثل `funlands.example` و `partner.example` و همه کلیدها و شناسه‌های این متن نمونه‌ان. مقدار واقعی هر محیط توی برگ شریک ثبت میشه.

وضعیت: مسیرهای این راهنما، یعنی JWKS فان‌لندز، ورود وب، bootstrap و refresh، خروج، توکن گزارش، گزارش‌ها و وب‌هوک، الان در حال پیاده‌سازی هستن و هنوز روی sandbox باز نشدن. وقتی sandbox آماده شد نشانی‌های واقعی رو توی برگ شریک می‌فرستیم. ورود اپ بومی، حذف حساب و API رویدادهای فنی بازی هنوز طراحی نهایی ندارن و توی بخش ۱۲ اومدن.

تغییرات نسخه ۱.۱ (۱۶ آبان ۱۴۰۵): فرم launch و bootstrap حالا روی Origin بازی فان‌لندز (`FUNLANDS_GAME_ORIGIN`) نوشته شدن، نه روی نشانی پایه API؛ کلید `x-partner-api-key` برای درخواست‌های صفحه بازی و خطای `PARTNER_TOKEN_MISMATCH` اضافه شد؛ کد refresh بعد از خروج اصلاح شد (بخش ۴.۷)؛ و روشن شد که `funlands:relaunch_requested` رو خود صفحه بازی می‌فرسته (بخش ۴.۹).

۱. اتصال در یک نگاه

اتصال شریک به فان‌لندز روی این چند قاعده ساخته شده:

- هر کار توکن خودش رو داره: توکن launch با عمر حداکثر ۱۲۰ ثانیه، access بازی ۱۵ دقیقه، refresh چرخشی با سقف ۱۲ ساعت، و یک توکن جدا برای گزارش. هیچ توکن بلندمدتی برای کاربر صادر نمیشه.
- هر طرف فقط نشانی JWKS خودش رو ثبت می‌کنه. نه فایل کلید رد و بدل میشه نه سکرت مشترک، و چرخش و ابطال کلید روال مشخص داره.

- ورود وب با یک فرم POST داخل iframe انجام میشه و یک verifier کد bootstrap رو به همون پنجره گره می‌زنه. هیچ توکن یا کدی توی URL نمیره.
- رویدادها با وب‌هوک امضاشده در قالب SET می‌رسن، شناسه `event_id` ثابته، تا ۷۲ ساعت دوباره فرستاده میشن و یک API هم برای گرفتن رویدادهای جامونده هست.
- خروج کاربر با یک پیام امضاشده از سرور شریک انجام میشه و اثرش فوریه.
- خطاها کد پایدار دارن و برای هر کد گفتیم شریک چی کار کنه.

نمونه پیاده‌سازی همه این کارها در سمت شریک، با NestJS، توی بخش ۱۰ اومده.

چند اصطلاح

اصطلاح	معنی در این راهنما
شریک، محیط شریک	سرویسی که کاربر خودش رو احراز هویت می‌کنه و بازی رو نشون میده. برای هر شریک در هر محیط (sandbox یا production) یک رکورد جدا ثبت می‌کنیم.
`issuer`	شناسه دقیق صادرکننده توکن، به شکل نشانی HTTPS. باید حرف‌به‌حرف با مقدار ثبت‌شده یکی باشه.
JWKS، `kid`	فایل JSON عمومی که کلیدهای عمومی امضا رو منتشر می‌کنه. هر کلید یک `kid` یکتا داره و توکن با همین `kid` کلیدش رو معرفی می‌کنه.
`sub`	شناسه مستعار و پایدار کاربر که فقط مخصوص اتصال همین شریک به فان‌لندزه.
`sid`	شناسه نشست کاربر در سمت شریک. برای خروج دقیق لازمه.
برگ شریک	فرمی که مقدارهای هر محیط، مثل issuer و JWKS و Originها و وب‌هوک، با مالک مشخص توش ثبت میشه.

۲. شروع همکاری

۲.۱ چیزهایی که از شریک می‌گیریم

این موارد برای sandbox و production جدا پر میشن.

مورد	توضیح	تایید با
نام شریک، `pid` و محیط‌ها	نام حقوقی و شناسه فنی، و اینکه به کدوم محیط‌ها نیاز دارید	مالک محصول و امنیت
`issuer` و نشانی JWKS	نشانی دقیق HTTPS برای هر محیط، الگوریتم امضا، کلیدهای آزمایشی و برنامه چرخش کلید	مالک امنیت
Originهای وب و کلاینت‌های بومی	فهرست دقیق Originهایی که صفحه بازی رو باز می‌کنن (حداکثر ۳۲ تا)، و فهرست اپ‌های iOS، Android و TV با روش handoff هر کدوم	مالک کلاینت
قالب `sub` ، `sid` و `age_band`	پایداری و طول شناسه، مجموعه مقدارهای رده سنی و سیاست سن	محصول و حریم خصوصی
وب‌هوک و مخاطب حادثه	نشانی وب‌هوک هر محیط، و یک کانال احرازشده برای اعلام حادثه امنیتی که به کلیدهای امضا وابسته نباشه	عملیات و امنیت

`client_id` و دسترسی گزارش	کلیدی که درخواست توکن گزارش رو امضا می‌کنه (از همون JWKS یا یک JWKS جدا)، scopeهای لازم و نقش‌های پنل	مالک داده
منطقه زمانی و شاخص‌ها	تعریف «روز» برای گزارش، cohort، تاخیر قابل قبول داده و اینکه کدوم عدد مبنای تجاری میشه	محصول و مالی
سیاست نگهداری و حذف	مدت نگهداری داده خام، export، feed و backup، و مهلت کامل شدن حذف حساب	حقوقی و حریم خصوصی

۲.۲ چیزهایی که به شریک می‌دیم

مورد	توضیح
`partner_id`	شناسه داخلی شریک در فان‌لندز. توی پاسخ‌ها و پنل می‌بینیدش، ولی هیچ‌وقت لازم نیست توی درخواست بفرستیدش. ما شریک رو فقط از توکن امضاشده تشخیص می‌دیم.
`client_id`	شناسه کلاینت برای گرفتن توکن گزارش، جدا برای هر محیط
audienceها	سه مقدار جدا برای هر محیط که موقع ثبت توی برگ شریک بهتون میدیم: ورود (`…/partner-launch`)، خروج (`…/partner-logout`) و token endpoint (`…/oauth/token`). مقایسه حرف‌به‌حرفه، پس همون مقدار برگ شریک رو بذارید.
JWKS فان‌لندز و issuer ما	نشانی `/.well-known/jwks.json` هر محیط و مقدار `iss` که روی SETها می‌ذاریم
نشانی‌های sandbox	launch، خروج، token، پایه API گزارش و feed رویداد
Origin بازی (`FUNLANDS_GAME_ORIGIN`)	Origin دقیقی که صفحه handoff و صفحه بازی روش باز میشن، مثلا `https://play.sandbox.funlands.example` ، جدا برای هر محیط. فرم launch به همین Origin میره و CSP، iframe و چک `postMessage` هم با همین مقدار تنظیم میشن. توی بعضی استقرارها ممکنه با نشانی پایه API یکی باشه، ولی همیشه مقدار برگ شریک رو بذارید. JWKS، خروج، token، گزارش و feed روی نشانی پایه API می‌مونن.
`x-partner-api-key`	فقط برای شریک‌های `client_direct` . کلیدی که شریک رو معرفی می‌کنه و صفحه بازی همراه درخواست‌هاش می‌فرسته (بخش ۴.۷). سکرت کاربر نیست، ولی فقط یک بار نشون داده میشه و با چرخیدنش کلید قبلی باطل میشه.
قرارداد API و رویداد	OpenAPI نسخه‌دار، schema رویدادها و نشانی نوع هر رویداد
آزمون انطباق	مجموعه تستی که شریک روی sandbox اجرا می‌کنه. فهرستش توی بخش ۱۱ هست.
دسترسی پنل	دعوت کاربرهای پنل با نقش‌های analyst، owner و support، همراه با MFA

۲.۳ جدایی sandbox و production

- برای هر محیط یک رکورد جدا داریم با JWKS، issuer، الگوریتم، audienceها، Originها، وب‌هوک، `client_id` ، داده و پنل خودش.
- کلید امضای sandbox و production هیچ‌وقت یکی نیست. توکن sandbox روی production با `AUDIENCE_MISMATCH` رد میشه و برعکس.

- توی sandbox فقط داده غیرواقعی بفرستید. شناسه کاربر واقعی رو برای تست استفاده نکنید.
- بازیکن‌ها برای هر محیط جدا ساخته میشن: اگه یک `sub` هم توی sandbox و هم توی production وارد بشه، دو حساب جدا در فان‌لندز داره و داده بازی، نشست‌ها و گزارش‌هاش هیچ‌وقت قاطی نمیشن.

۲.۴ مسیر رسیدن به production

۱. شریک برگ بخش ۲.۱ رو برای sandbox پر می‌کنه و از یک کانال اداری احرازشده برامون می‌فرسته. نشانی JWKS رو فقط از همین کانال ثبت می‌کنیم، نه از داخل توکن.

۲. ما رکورد sandbox رو می‌سازیم، JWKS شما رو یک بار می‌خونیم و چک می‌کنیم کلیدها درست باشن، الگوریتم مجاز باشه و هر کلید `kid` داشته باشه. بعد مقادیر بخش ۲.۲ رو می‌فرستیم.

۳. شریک آزمون انطباق رو روی sandbox اجرا می‌کنه تا همه موارد پاس بشن، و بعد ترافیک آزمایشی با داده غیرواقعی می‌فرسته.

۴. مالک امنیت دو طرف تست‌های منفی توکن و تمرین چرخش کلید رو امضا می‌کنن. برگ داده، فهرست رویدادها، تعریف شاخص‌ها و روال حادثه هم با مالک مشخص تایید میشن.

۵. رکورد production با کلیدهای جدید و مقدارهای جدا ثبت میشه و ترافیک واقعی اول به‌صورت محدود باز میشه.

۳. کلیدها

۳.۱ قاعده‌های اصلی

- هر طرف با کلید خصوصی خودش امضا می‌کنه. کلید خصوصی توی KMS یا HSM همون سازمان می‌مونه و به هیچ‌کس داده نمیشه، نه از طریق API، نه با ایمیل، نه «دستی» برای راه افتادن سریع. ما هم هیچ‌وقت چنین چیزی نمی‌خوایم. اگه کسی از طرف فان‌لندز کلید خصوصی خواست، این خودش یک حادثه امنیتیه.
- سکرت مشترک هم نداریم. الگوریتم `none` و کل خانواده `HS256` / `HS384` / `HS512` رد میشن.
- کلید عمومی رو فقط از نشانی JWKS ثبت‌شده همون issuer و همون محیط می‌خونیم. هدرهای `jku`، `x5u` و `jwk` منبع کلید نیستن و توکنی که هر کدومشون رو داشته باشه رد میشه.
- الگوریتم رو از برگ شریک برمی‌داریم، نه از هدر توکن. برای هر محیط یک الگوریتم ثابت ثبت میشه. پیشنهاد ما `ES256` (منحنی P-۲۵۶) هست. `RS256`، `PS256` و `EdDSA` هم پشتیبانی میشن، و برای RSA کلید باید دست‌کم ۲۰۴۸ بیت باشه. عوض کردن الگوریتم چرخش کلید حساب نمیشه، تغییر سیاسته و تست دوباره می‌خواد.
- اختلاف ساعت مجاز برای همه توکن‌ها ۳۰ ثانیه‌ست. ساعت سرورها رو با NTP هماهنگ نگه دارید.

۳.۲ شریک JWKS

نشانی JWKS باید HTTPS باشه، بدون ریدایرکت جواب بده و فقط کلید عمومی داشته باشه. وجود `d`، `p`، `q` یا هر پارامتر خصوصی دیگه خطای امنیتیه و اون JWKS رو قبول نمی‌کنیم. هر کلید باید `kid`، `alg` و `use: "sig"` داشته باشه.

```
GET https://auth.sandbox.partner.example/.well-known/jwks.json

{
  "keys": [{
    "kty": "EC", "crv": "P-256", "use": "sig", "alg": "ES256",
    "kid": "partner-a-sandbox-2026-11",
    "x": "<base64url-public-x>", "y": "<base64url-public-y>"
  }]
}
```

رفتار ما موقع خوندن JWKS شما:

- فقط نشانی ثبت‌شده رو می‌خونیم. ریدایرکت رو دنبال نمی‌کنیم و به آدرس‌های داخلی یا خصوصی وصل نمیشیم.
- timeout سه ثانیه‌ست، پاسخ حداکثر ۶۴ کیلوبایت و حداکثر ۲۰ کلید. JWKS رو ۵ دقیقه کش می‌کنیم.
- اگه توکنی با `kid` ناشناخته برسه، یک بار JWKS رو دوباره می‌خونیم (برای هر محیط حداکثر یک بار در هر ۳۰ ثانیه). اگه باز هم کلید نبود، توکن با `KEY_NOT_FOUND` رد میشه.
- اگه JWKS در دسترس نباشه، فقط کلیدهایی که قبلا معتبر و کش شده بودن تا حداکثر یک ساعت قبول میشن. کلید ناشناخته یا کلید ابطال‌شده به هیچ وجه به خاطر قطعی قبول نمیشه.

۳.۳ قاعده‌های `kid`

- `kid` باید رشته غیرخالی باشه و توی issuer و محیط شما یکتا باشه. لازم نیست بین شریک‌ها یکتا باشه، چون ما کلید رو با جفت issuer و `kid` پیدا می‌کنیم.
- `kid` یک کلید قدیمی رو هیچ‌وقت برای کلید جدید استفاده نکنید.
- پیشنهاد قالب: `<نام شریک>-<محیط>-<سال-ماه>`، مثلا `partner-a-sandbox-2026-11`.

۳.۴ چرخش برنامه‌ریزی‌شده کلید شریک

۱. کلید جدید K۲ رو با `kid` تازه توی KMS بسازید. هنوز با K۱ امضا کنید.

۲. K۱ و K۲ رو با هم توی JWKS منتشر کنید.

۳. دست‌کم ۲۴ ساعت صبر کنید. حداقل فنی بیشینه کش ما به‌اضافه حاشیه‌ست، ولی ۲۴ ساعت جلوی مشکلات کش‌های میانی رو هم می‌گیره.

۴. همه توکن‌ها رو با K۲ امضا کنید: launch، logout و client assertion. ما کلید رو با `kid` انتخاب می‌کنیم و در این مدت هر دو کلید رو قبول می‌کنیم.

۵. K۱ رو دست‌کم ۲۴ ساعت بعد از آخرین امضا با اون توی JWKS نگه دارید، بعد حذفش کنید. حداقل فنی بلندترین عمر توکن شما (۱۲۰ ثانیه) به‌اضافه ۳۰ ثانیه اختلاف ساعت و زمان کشه.

۶. پیشنهاد ما اینه که کلیدها رو دست‌کم هر ۱۸۰ روز عوض کنید و هر بار چرخش رو اول روی sandbox تمرین کنید.

۳.۵ چرخش برنامه‌ریزی‌شده کلید فان‌لندز

۱. کلید جدید ما دست‌کم ۲۴ ساعت قبل از اولین استفاده توی JWKS فان‌لندز منتشر میشه. JWKS ما `Cache-Control: public, max-age=300` برمی‌گردونه.

۲. بعد SETهای جدید با کلید جدید امضا میشن. رویدادهایی که دوباره فرستاده میشن هم امضای تازه با کلید جدید می‌گیرن ولی `event_id` قبلی رو نگه می‌دارن.

۳. کلید قدیمی دست‌کم ۲۴ ساعت بعد از آخرین امضا توی JWKS می‌مونه.

۴. سمت شما: کلید ما رو توی کد ثابت نکنید. با `kid` از JWKS ما انتخاب کنید، کش رو حداکثر ۱۰ دقیقه نگه دارید و برای `kid` ناشناخته یک بار و با محدودیت نرخ دوباره بخونید.

۳.۶ ابطال اضطراری، وقتی کلید شریک لو رفته

حذف کردن کلید از JWKS به‌تنهایی کافی نیست، چون نسخه کش‌شده تا چند دقیقه هنوز اون کلید رو داره. ابطال واقعی با اعلام مستقیم و denylist انجام میشه.

۱. امضا با K۱ رو همون لحظه متوقف کنید و K۱ رو توی KMS غیرفعال کنید.

۲. از کانال حادثه‌ای که توی برگ شریک ثبت شده به ما خبر بدید: issuer، محیط، `kid` لو رفته و بازه زمانی‌ای که کلید در خطر بوده. این پیام نباید با کلید لو رفته امضا یا تایید بشه.

۳. ما جفت issuer و `kid` رو برای اون محیط فورا توی denylist می‌ذاریم. denylist بر کش مقدمه، پس هر توکن K۱، حتی با `exp` معتبر، رد میشه.

۴. نشست‌هایی که توی بازه خطر ساخته شدن رو طبق روال حادثه باطل می‌کنیم. اگه دامنه نشت روشن نباشه، ورود جدید اون محیط رو موقتا می‌بندیم.

۵. K۲ رو با `kid` تازه منتشر کنید و K۱ رو از JWKS بردارید. با اولین توکن K۲ ما JWKS رو دوباره می‌خونیم.

۶. بعد از حادثه یک گزارش کوتاه رد و بدل می‌کنیم. اگه همون کلید برای client assertion گزارش هم استفاده میشده، اون هم باید عوض بشه.

۳.۷ ابطال اضطراری، وقتی کلید فان‌لندز لو رفته

۱. ما امضا با K۱ و در صورت لزوم ارسال وب‌هوک رو متوقف می‌کنیم و K۲ رو فعال می‌کنیم.

۲. از کانال حادثه ثبت‌شده به شما خبر می‌دیم، با `kid` لو رفته و زمان.

۳. شما `kid` رو فورا توی denylist خودتون بذارید و هر SET که با K۱ امضا شده رو رد کنید، حتی اگه تازه باشه.

۴. بعد از رفع مشکل، رویدادهای تحویل‌نشده با امضای تازه و همون `event_id` دوباره ارسال میشن.

۴. ورود بازی در وب

۴.۱ مراحل در یک نگاه

۱. کاربر توی سرویس شما وارد شده و شما چک کردید که این بازی براش مجازه.

۲. سرور شما یک `bootstrap_verifier` تصادفی تازه می‌سازه و `launch_challenge` رو ازش حساب می‌کنه.

۳. سرور شما توکن launch رو با کلید خصوصی امضا می‌کنه. توکن هیچ‌وقت توی مرورگر یا اپ ساخته نمیشه.

۴. صفحه شما یک iframe و یک فرم مخفی داره که `launch_token` ، `bootstrap_verifier` و `game` رو با POST به `/v1/partner/launch` روی Origin بازی فان‌لندز (`FUNLANDS_GAME_ORIGIN`) داخل همون iframe می‌فرسته.

۵. فان‌لندز درخواست رو چک می‌کنه، بازیکن رو پیدا می‌کنه یا می‌سازه، یک نشست می‌سازه و یک صفحه HTML برمی‌گردونه.

۶. اون صفحه داخل iframe کد bootstrap رو با همون verifier عوض می‌کنه و توکن‌های بازی رو می‌گیره. این قدم و refresh کار خود صفحه بازیه و سرور شما کاری نمی‌کنه.

۴.۲ توکن launch

هدر:

فیلد	قاعده
`typ`	دقیقا `funlands-launch+jwt` . یک `id_token` معمولی OIDC یا access token شما توکن launch حساب نمیشه.
`alg`	همون الگوریتم ثبت‌شده برای این محیط
`kid`	رشته غیرخالی که توی JWKS ثبت‌شده شما هست

ممنوع	`jku` ، `x5u` و `jwk`

claimها:

claim	اجباری	قاعده
`iss`	بله	حرف‌به‌حرف برابر issuer ثبت‌شده همین محیط
`aud`	بله	audience ورود همین محیط، مثلا `https://sandbox.funlands.example/partner-launch`
`sub`	بله	شناسه مستعار، پایدار و مخصوص همین اتصال، حداکثر ۱۲۸ کاراکتر. ایمیل، شماره تلفن، کد ملی یا شناسه سراسری قابل ردیابی نباشه. اگه `sub` عوض بشه از نظر ما یک بازیکن جدیده.
`sid`	بله	شناسه نشست کاربر در سمت شما. خروج دقیق با همین انجام میشه.
`iat` ، `exp`	بله	`exp - iat` حداکثر ۱۲۰ ثانیه. `iat` بیشتر از ۳۰ ثانیه در آینده نباشه.
`jti`	بله	یکتا و یک‌بارمصرف، مثلا UUID
`game`	بله	شناسه بازی. باید دقیقا با فیلد `game` فرم یکی باشه و بازی برای شما فعال باشه. اگه ورود به لابی بخواید، مقدارش رو با هم توافق می‌کنیم.
`launch_challenge`	بله	`base64url(SHA-256(bootstrap_verifier))`
`locale`	نه	BCP ۴۷، مثلا `fa-IR`
`age_band`	نه	رده سنی از مجموعه مقدارهای توافق‌شده در برگ شریک. اگه نباشه رده سنی ناشناخته حساب میشه و محتوای محدودشده نشون داده نمیشه.
`nickname`	نه	نام نمایشی. فیلتر میشه و حداکثر ۲۴ کاراکتره.

نمونه هدر و payload (مقدارها نمونه‌ان):

```
{ "alg": "ES256", "typ": "funlands-launch+jwt", "kid": "partner-a-sandbox-2026-11" }

{
  "iss": "https://auth.sandbox.partner.example",
  "aud": "https://sandbox.funlands.example/partner-launch",
  "sub": "pw_7f3c9a1e",
  "sid": "partner_session_123",
  "iat": 1793440800,
  "exp": 1793440920,
  "jti": "2f6d1c1e-0b7a-4c55-9a43-5b1f0e8d9c21",
  "game": "star-runner",
  "launch_challenge": "<base64url-sha256-of-bootstrap-verifier>",
  "locale": "fa-IR",
  "age_band": "13-17"
}
```

۴.۳ verifier و ساخت توکن

برای هر ورود یک verifier تازه بسازید و هیچ‌وقت تکرارش نکنید. نمونه کوتاه با Node.js و کتابخونه jose اینه و نسخه کامل NestJS توی بخش ۱۰.۳ هست:

```
import { randomBytes, createHash, randomUUID } from 'node:crypto';
import { SignJWT } from 'jose';

const verifier  = randomBytes(32).toString('base64url');
const challenge = createHash('sha256').update(verifier, 'ascii').digest('base64url');

const now = Math.floor(Date.now() / 1000);   // one clock read: exp - iat stays exactly 120
const launchToken = await new SignJWT({ sid, game, launch_challenge: challenge })
  .setProtectedHeader({ alg: 'ES256', typ: 'funlands-launch+jwt', kid: ACTIVE_KID })
  .setIssuer(ISSUER)
  .setAudience(LAUNCH_AUDIENCE)
  .setSubject(pairwiseSub)
  .setIssuedAt(now)
  .setExpirationTime(now + 120)
  .setJti(randomUUID())
  .sign(signer);   // production: sign inside KMS or HSM
```

۴.۴ صفحه شریک و درخواست

```
<iframe name="game-frame" title="Game"></iframe>
<form method="post" action="https://play.sandbox.funlands.example/v1/partner/launch" target="game-
frame">
  <input type="hidden" name="launch_token"       value="{{escapedToken}}">
  <input type="hidden" name="bootstrap_verifier" value="{{escapedVerifier}}">
  <input type="hidden" name="game"               value="{{escapedGameId}}">
</form>
```

```
POST /v1/partner/launch
Host: play.sandbox.funlands.example
Content-Type: application/x-www-form-urlencoded
Origin: https://sandbox.partner.example

launch_token=<signed-jwt>&game=star-runner&bootstrap_verifier=<base64url-32-random-bytes>
```

- توکن و verifier هیچ‌وقت توی query string یا fragment نمیان و نباید لاگ بشن. صفحه‌ای که اون‌ها رو داره باید `Cache-Control: no-store` داشته باشه.
- Origin صفحه باید دقیقا یکی از Originهای ثبت‌شده باشه. درخواست بدون Origin یا با Origin برابر `null` رد میشه. Origin به‌تنهایی احراز هویت نیست و فقط یک لایه کنترله.
- فرم رو به Origin بازی (`FUNLANDS_GAME_ORIGIN`) بفرستید، نه به نشانی پایه API. صفحه handoff کد رو روی همون Origin با bootstrap عوض می‌کنه و bootstrap فقط درخواستی رو قبول می‌کنه که Origin اون دقیقا Origin بازی باشه؛ اگه فرم به Origin دیگه‌ای بره، ورود با `ORIGIN_NOT_ALLOWED` رد میشه.
- CSP صفحه خودتون رو طوری تنظیم کنید که `frame-src` و `form-action` فقط Origin بازی فان‌لندز همون محیط رو مجاز کنن.
- `Referrer-Policy` صفحه خودتون رو `strict-origin-when-cross-origin` بذارید (پیش‌فرض مرورگرها)، همون مقداری که نمونه‌های این راهنما می‌ذارن.

هشدار: روی صفحه‌ای که فرم launch رو می‌فرسته `Referrer-Policy: no-referrer` نذارید. با این مقدار مرورگر توی POST بین دو دامنه هدر `Origin: null` می‌فرسته و ورود با `ORIGIN_NOT_ALLOWED` رد میشه.

۴.۵ چیزهایی که ما چک می‌کنیم

۱. حجم درخواست، `Content-Type`، محدودیت نرخ و Origin.

۲. هدر و `iss` رو فقط برای پیدا کردن رکورد شریک می‌خونیم. تا امضا تایید نشه به هیچ‌کدوم اعتماد نمی‌کنیم.

۳. `typ`، الگوریتم، `kid`، نبودن هدرهای ممنوع و denylist.

۴. امضا با کلید JWKS ثبت‌شده، بعد همه claimها، برابری `game` با فرم، و برابری `SHA-256(bootstrap_verifier)` با `launch_challenge`.

۵. یک‌بارمصرف بودن `jti` با یک رکورد ماندگار در دیتابیس. ساخت بازیکن، نشست و رویدادهای `player.created` و `session.started` همه توی یک تراکنش انجام میشه.

۴.۶ پاسخ launch و bootstrap

پاسخ موفق یک صفحه HTML با `Cache-Control: no-store`، `Referrer-Policy: same-origin` و CSP سخت‌گیرانه‌ست که `frame-ancestors` اون فقط Originهای ثبت‌شده شما رو مجاز می‌کنه. پس صفحه بازی رو فقط داخل صفحه‌های خودتون می‌تونید قاب بگیرید. این صفحه کد bootstrap و verifier رو فقط توی حافظه نگه می‌داره و بلافاصله این درخواست رو روی همون Origin بازی می‌فرسته، بعد توکن‌ها رو به صفحه بازی همین Origin تحویل میده و اون رو باز می‌کنه. این بخش برای تیم کلاینت فان‌لندزه و شریک نباید صداش بزنه. پاسخ موفق bootstrap پاکت `{ "state", "data" }` نداره و فیلدهاش snake_case هستن:

```
POST /v1/session/bootstrap
Host: play.sandbox.funlands.example
Origin: https://play.sandbox.funlands.example
Content-Type: application/json

{ "code": "boot_01J...", "bootstrap_verifier": "<same-verifier>" }

200 OK
Cache-Control: no-store

{ "access_token": "<funlands-jwt>", "expires_in": 900,
  "refresh_token": "<opaque-token>", "token_type": "Bearer" }
```

چیز	عمر	نکته
توکن launch	حداکثر ۱۲۰ ثانیه	یک‌بارمصرف، امضای شریک
کد bootstrap	۳۰ ثانیه	یک‌بارمصرف، فقط با verifier درست. کد اشتباه، verifier اشتباه و کد مصرف‌شده همه یک خطای یکسان می‌گیرن.
access token بازی	حداکثر ۱۵ دقیقه	مخصوص سرویس‌های بازی فان‌لندز. برای گزارش قبول نمیشه و شریک نباید بهش تکیه کنه یا محتواش رو بخونه.
refresh token	سقف مطلق ۱۲ ساعت	opaque، با هر استفاده عوض میشه و فقط hash اون ذخیره میشه

refresh ۴.۷

صفحه بازی از همون endpoint تمدید نشست عادی فان‌لندز استفاده می‌کنه. بدنه و پاسخش camelCase هستن و پاسخ موفق داخل پاکت `{ "state": true, "data": ... }` میاد:

```
POST /v1/auth/session/refresh
Content-Type: application/json

{ "refreshToken": "<opaque-token>" }

200 OK
{ "state": true,
  "data": { "accessToken": "<funlands-jwt>", "refreshToken": "<rotated-opaque-token>",
            "accessTokenExpiresAt": "2026-11-05T09:45:00.000Z",
            "refreshTokenExpiresAt": "2026-11-05T21:30:00.000Z" } }
```

- هر refresh موفق یک access و یک refresh تازه میده و refresh قبلی مصرف میشه.
- توکن‌ها توی `localStorage` نگه داشته نمیشن. اگه صفحه reload بشه، ممکنه refresh از دست بره و لازم باشه ورود دوباره انجام بشه.
- refresh ناشناخته یا منقضی، یا refresh بعد از سقف ۱۲ ساعت، جواب `401 TOKEN_EXPIRED` می‌گیره.
- اگه یک refresh که قبلا چرخیده دوباره استفاده بشه، همه نشست‌های اون بازیکن باطل میشن و جواب `401 TOKEN_REUSE` هست؛ کاربر باید دوباره از طریق شما وارد بشه. پس دو refresh هم‌زمان با یک توکن نفرستید.
- بعد از خروج back-channel، فان‌لندز نشست رو همون موقع حذف می‌کنه، پس refresh معمولا جواب `401 TOKEN_EXPIRED` می‌گیره، همون کدی که access token اون نشست هم از این به بعد می‌گیره. `401 SESSION_REVOKED` فقط وقتی میاد که refresh قبل از حذف نشست برسه: مثلا وقتی خروج یا غیرفعال شدن محیط ثبت شده ولی بستن نشست هنوز تموم نشده (خروجی که `503` گرفته و هنوز تکرار نشده). صفحه بازی هر دو کد رو یعنی «نشست تموم شده» حساب می‌کنه و ورود دوباره می‌خواد (بخش ۴.۹)؛ دوباره refresh نمی‌کنه.

برای شریک‌های `client_direct`، صفحه بازی باید روی هر درخواست HTTP به APIهای بازی هدر `x-partner-api-key` رو بفرسته و روی هر اتصال WebSocket همین کلید رو با هدر `x-partner-api-key` یا `auth.partnerApiKey` در handshake بده. access token یک بازیکن شریک به همون شریک بسته‌ست؛ درخواستی که این کلید رو نداره فضای پلتفرم حساب میشه و با `403` `PARTNER_TOKEN_MISMATCH` رد میشه. خود refresh به این کلید نیاز نداره، ولی کلید نامعتبر یا چرخیده روی هر درخواستی جواب `401` `PARTNER_KEY_INVALID` می‌گیره.

- این کلید رو فان‌لندز برای هر شریک `client_direct` صادر می‌کنه و توی برگ شریک تحویل میده. فقط شریک رو معرفی می‌کنه، سکرت کاربر نیست و توی کد مرورگر دیده میشه. با این حال لاگش نکنید و توی URL نذارید. با چرخیدنش کلید قبلی همون لحظه باطل میشه.
- شریک‌های `server_to_server` با `X-Partner-Public-Key` و `X-Partner-Secret` کار می‌کنن. این سکرت هیچ‌وقت نباید به مرورگر، اپ یا صفحه بازی برسه.
- اینکه فضای شریک از خود توکن بازیکن شریک خونده بشه و این هدر لازم نباشه در حال بررسیه و هنوز تصمیمی براش گرفته نشده (بخش ۱۲). تا اون موقع هدر رو بفرستید.

۴.۸ تکرار درخواست ورود

- اگه همون توکن با همون محتوا و همون verifier درست دوباره برسه، مثلا چون شبکه قطع شده، نشست دوم ساخته نمیشه و یک کد bootstrap تازه برای همون نشست برمی‌گرده. این کار حداکثر سه بار و فقط تا قبل از `exp` ممکنه.
- اگه همون `jti` بدون verifier درست برسه، جواب `TOKEN_REPLAYED` هست.
- اگه همون `jti` با محتوای متفاوت برسه، جواب `409 LAUNCH_PAYLOAD_CHANGED` هست.
- دو ورود هم‌زمان با یک `sub` توی یک محیط همیشه به یک بازیکن می‌رسن.

۴.۹ ورود دوباره و خطای ورود با postMessage

توی iframe دو صفحه مختلف ممکنه به صفحه شما پیام بدن، هر دو روی Origin بازی. صفحه handoff فان‌لندز (پاسخ launch) فقط `funlands:launch_failed` رو می‌فرسته. `funlands:relaunch_requested` رو خود صفحه بازی می‌فرسته، وقتی نشستی نداره: مثلا بعد از reload، یا وقتی refresh با `TOKEN_EXPIRED` یا `SESSION_REVOKED` رد شده. توی پیام‌ها هیچ توکن یا `sub` ی نمیاد.

```
// از صفحه بازی به صفحه شریک
{ "type": "funlands:relaunch_requested", "request_id": "r_123" }

// تازه از سرور خودتون launch از صفحه شریک به صفحه بازی، بعد از گرفتن
{ "type": "partner:relaunch_ready", "request_id": "r_123" }

// از صفحه handoff، اگه ورود یا bootstrap رد بشه
{ "type": "funlands:launch_failed", "code": "TOKEN_EXPIRED", "request_id": "req_01J..." }
```

- پیام رو فقط با `targetOrigin` دقیق Origin بازی (`FUNLANDS_GAME_ORIGIN`) همون محیط بفرستید. موقع دریافت، `event.origin` رو با همین مقدار و `event.source` رو با پنجره iframe بسنجید، و برای `partner:relaunch_ready` همون `request_id` پیام درخواست رو برگردونید.
- `request_id` توی `funlands:launch_failed` شناسه درخواست launch در فان‌لندزه (همون `X-Correlation-Id`)، نه شناسه درخواست relaunch. اون رو لاگ کنید و موقع تماس با ما بفرستید. اگه launch از Originی بیاد که ثبت نشده، صفحه handoff ساخته نمیشه، پیامی هم نمیاد و iframe فقط بدنه خطای JSON رو نشون میده.
- بعد از `partner:relaunch_ready` ، فرم ورود رو با توکن و verifier تازه دوباره داخل iframe ارسال کنید.
- اگه سرور شما در دسترس نیست، یک پیام قابل فهم با دکمه تلاش دوباره نشون بدید.

۵. اپ‌های بومی (iOS، Android و TV)

مسیر ورود اپ بومی هنوز نهایی نشده و فعلا فقط ورود وب رو باز می‌کنیم. WebView معمولا Origin قابل اعتماد نمی‌فرسته، پس صرفا برداشتن چک Origin و گذاشتن محدودیت نرخ مجوز ورود نیست. تا وقتی مسیر زیر تصویب و پیاده‌سازی نشده، هیچ مسیر بدون Origin باز نمی‌کنیم. چیزی که برای این مسیر لازم میشه:

- سرور شما ورود رو به‌صورت سرور به سرور برای ما می‌فرسته، با احراز هویت کلاینت یا امضای جدا و یک `nonce` .
- ما یک کد handoff کوتاه‌عمر برمی‌گردونیم که به همون دستگاه یا نمونه WebView و به `nonce` گره خورده.
- اپ فقط همون کد رو توی WebView باز می‌کنه و ما یک بار مصرفش می‌کنیم. توکن کاربر شما نباید به اپ داده بشه که اپ با اون APIهای مختلف رو صدا بزنه.
- روش اثبات اپ، روش گره زدن به دستگاه و رفتار در قطع شبکه برای iOS، Android و TV هر کدوم جدا بررسی و تایید میشن. User-Agent، IP یا نبودن Origin اثبات حساب نمیشن.

اگه اپ بومی دارید، فهرست اپ‌ها و نسخه‌ها رو توی برگ شریک بنویسید تا این بخش رو با شما نهایی کنیم.

۶. خروج و حذف حساب

۶.۱ خروج از سمت شریک

وقتی کاربر از سرویس شما خارج میشه یا نشستش رو باطل می‌کنید، سرور شما یک توکن خروج امضاشده می‌فرسته:

```
POST /partner/v1/backchannel-logout
Content-Type: application/x-www-form-urlencoded

logout_token=<signed-partner-logout-jwt>
```

```
{ "alg": "ES256", "typ": "logout+jwt", "kid": "partner-a-sandbox-2026-11" }

{
  "iss": "https://auth.sandbox.partner.example",
  "aud": "https://sandbox.funlands.example/partner-logout",
  "sub": "pw_7f3c9a1e",
  "sid": "partner_session_123",
  "iat": 1793444400,
  "exp": 1793444520,
  "jti": "9b0f6a52-6a1e-4f0e-8f63-1d2b7c4e5a10",
  "events": { "http://schemas.openid.net/event/backchannel-logout": {} }
}
```

- `typ` باید `logout+jwt` باشه و `aud` audience مخصوص خروج. توکن launch رو برای خروج نفرستید و برعکس، چون رد میشه.
- `exp` اجباریه و `exp - iat` حداکثر ۱۲۰ ثانیه‌ست. `nonce` نباید باشه.
- `sid` اجباریه. خروج همه نشست‌های یک `sub` بدون `sid` فقط با توافق جدا فعال میشه.
- پاسخ `200` یعنی دستور ثبت و اجرا شده یا قبلا اجرا شده بوده. ارسال دوباره همون `jti` هم `200` می‌گیره. پس اگه جواب نگرفتید با خیال راحت تکرار کنید.

اثر خروج فوریه: نشست و کل خانواده refresh باطل میشن، اتصال‌های WebSocket بسته میشن و APIهای حساس وضعیت نشست رو در هر درخواست چک می‌کنن. پس لازم نیست منتظر تموم شدن access token پونزده‌دقیقه‌ای بمونید. بعد از پاسخ `200`، صفحه بازی روی access token و refresh همون نشست `401 TOKEN_EXPIRED` می‌گیره، چون نشست حذف شده؛ بخش ۴.۷ رو ببینید.

۶.۲ حذف حساب

حذف حساب دو وضعیت جدا داره: «درخواست دریافت شد» و «حذف کامل شد». این دو رو یکی حساب نکنید.

۱. شما درخواست حذف رو با یک `request_id` پایدار، `sub` و زمان درخواست می‌فرستید. ارسال دوباره همون `request_id` اثر دوم نداره.

۲. ما درخواست رو ماندگار ثبت می‌کنیم، همه نشست‌های اون بازیکن رو می‌بندیم و پردازش رو شروع می‌کنیم. جواب در این مرحله فقط یعنی «دریافت شد».

۳. ارتباط هویت و داده شخصی طبق قرارداد داده حذف یا ناشناس میشه. فقط یک رکورد حداقلی برای جلوگیری از تکرار و تایید نگه می‌داریم. آمارهای تجمیعی که واقعا شخصی نیستن طبق توافق می‌مونن.

۴. فقط بعد از کامل شدن حذف، رویداد `player.deleted` رو براتون می‌فرستیم. این رویداد مدرک کامل شدنه.

نشانی دقیق این درخواست و مهلت کامل شدن حذف هنوز نهایی نیست (بخش ۱۲). تا اون موقع اگه کاربری حسابش رو حذف کرد، دست‌کم توکن خروج رو بفرستید تا نشست‌هاش بسته بشن.

۷. رویدادها و وب‌هوک

۷.۱ فهرست رویدادها

هر رویداد این فیلدها رو داره: `event_id` ثابت، `schema_version`، `subject`، `player_seq` که برای هر بازیکن یکی‌یکی زیاد میشه، و `occurred_at` به وقت UTC با دقت ثانیه. نوع هر رویداد یک URI به شکل `https://api.funlands.ir/events/<type>` هست که توی هر دو محیط یکسانه. فهرست دقیق فیلدهای هر نوع توی schema نسخه ۱ میاد.

رویداد	کِی	فیلدهای اصلی
`player.created`	اولین ورود یک `sub`	`locale`، `age_band`
`session.started`	هر ورود موفق	`game_id`، `platform`
`session.ended`	پایان نشست؛ فعلا وقتی نشست با خروج back- channel شما بسته بشه	`duration_ms`، `games_played`
`game.started`	شروع یک دور یا مرحله	`game_id`، `level` در صورت توافق
`game.completed`	پایان معتبر که سرور تاییدش کرده (امتیازی که دور رو می‌بنده)، نه صرفا بسته شدن کلاینت	`game_id`، `game_version`، `score`، `duration_ms`، `result`
`achievement.unlocked`	باز شدن یک دستاورد	`achievement`
`player.suspended`	تعلیق بازیکن توسط فان‌لندز	`reason_code`
`player.deleted`	بعد از کامل شدن حذف	`request_id`

اگه محاسبه عددی بعدا اصلاح بشه، رویداد اصلاحی جدا می‌فرستیم و عدد تاریخی رو بی‌صدا عوض نمی‌کنیم.

فعلا این‌ها فرستاده میشن: `player.created`، `session.started`، `session.ended`، `game.completed` و `player.deleted`. `game.started`، `achievement.unlocked`، `player.suspended` و رویداد اصلاحی هنوز فرستاده نمیشن و وقتی بیان همین قالب رو دارن. این فیلدها فعلا همیشه `null` هستن: `games_played`، `game_version` و `duration_ms` در `game.completed`، و `request_id`. کد شما باید `null` رو قبول کنه.

۷.۲ شکل درخواست وب‌هوک

هر تلاش ارسال یک SET تازه طبق RFC ۸۴۱۷ هست که با کلید فان‌لندز امضا شده. `jti` و `iat` در هر تلاش عوض میشن ولی `event_id` داخل رویداد ثابت می‌مونه.

```
POST /funlands/events          (نشانی وب‌هوک ثبت‌شده شما)
Content-Type: application/secevent+jwt

<newly-signed-SET-for-this-attempt>
```

```
{ "alg": "ES256", "typ": "secevent+jwt", "kid": "funlands-sandbox-2026-11" }

{
  "iss": "https://sandbox-api.funlands.ir",
  "aud": "https://auth.sandbox.partner.example",
  "iat": 1793440860,
  "jti": "dlv_0193a0e2c4d57e6f8a90b1c2d3e4f5a6",
  "events": {
    "https://api.funlands.ir/events/game.completed": {
      "event_id": "evt_0193a0e1b2c37d4e8f50a1b2c3d4e5f6",
      "schema_version": 1,
      "subject": { "format": "iss_sub", "iss": "https://auth.sandbox.partner.example", "sub":
"pw_7f3c9a1e" },
      "player_seq": 42,
      "occurred_at": "2026-10-31T10:00:00Z",
      "game_id": "star-runner", "game_version": null, "score": 1250, "duration_ms": null, "result":
"win"
    }
  }
}
```

`iss` همون issuer فان‌لندز توی همین محیطه و `aud` همیشه issuer خود شماست. SET فیلد `exp` نداره.

۷.۳ چک کردن امضا در سمت شریک

۱. بدنه با نوع `application/secevent+jwt` رو به‌صورت متن و با سقف حجم بخونید.

۲. `typ` باید `secevent+jwt` باشه، الگوریتم فقط `ES256` و `kid` غیرخالی. توکنی که `jku` ، `x5u` یا `jwk` داره رو رد کنید. denylist خودتون رو هم چک کنید.

۳. کلید رو فقط از نشانی JWKS فان‌لندز که توی برگ شریک گرفتید بخونید.

۴. امضا، `iss` (issuer فان‌لندز همین محیط) و `aud` (issuer خود شما) رو با مقدارهای برگ شریک بسنجید، نه با چیزی که داخل درخواست اومده. `iat` حداکثر ۵ دقیقه قدیمی و حداکثر ۳۰ ثانیه آینده باشه.

۵. تکراری بودن رو با `event_id` بسنجید، نه با `jti` . رویداد تکراری هم باید جواب موفق بگیره.

۶. رویداد و رکورد پردازشش رو توی یک تراکنش ماندگار ذخیره کنید و بعد `202` بدید. پردازش رو بعدا انجام بدید. جواب رو حداکثر توی ۵ ثانیه بدید.

۷. ترتیب رسیدن تضمینی نیست، حتی برای یک بازیکن. آخرین `player_seq` هر بازیکن رو نگه دارید، فاصله رو علامت بزنید و از feed بازیابی پرش کنید.

```
const { payload } = await jwtVerify(body, funlandsJwks, {
  algorithms: ['ES256'], typ: 'secevent+jwt',
  issuer: FUNLANDS_SET_ISSUER, audience: OUR_AUDIENCE,
  requiredClaims: ['iss', 'aud', 'iat', 'jti', 'events'],
  clockTolerance: 30,
});
// then: check iat age (max 5 min), reject jku/x5u/jwk headers, dedupe on event_id
```

نسخه کامل این گیرنده با NestJS توی بخش ۱۰.۴ هست.

۷.۴ پاسخ شما و رفتار ما

پاسخ	معنی	کار ما

`200` یا `202`	ماندگار و idempotent ذخیره شد، از جمله رویداد تکراری	تحویل‌شده ثبت میشه. کد دیگه، حتی `204` ، موفق حساب نمیشه.
`400`	خطای دائمی schema	همون لحظه به dead letter میره و تکرار نمیشه؛ هشدار به هر دو طرف. بعد از رفع، replay میشه.
`401` یا `403`	مشکل امضا یا مجوز	هشدار و توقف کنترل‌شده ارسال به محیط شما. رویدادها توی صف می‌مونن. وقتی رفع مشکل رو به ما خبر دادید، ارسال رو از سر می‌گیریم: رویدادهای مونده دوباره فرستاده میشن و پنجره ۷۲ ساعته از نو شروع میشه.
`429`	محدودیت نرخ	تلاش دوباره با رعایت `Retry-After`
`5xx` ، timeout (بیشتر از ۵ ثانیه)، خطای شبکه، redirect یا هر کد دیگه	خطای گذرا	تلاش دوباره با backoff. redirect دنبال نمیشه.

جواب موفقی که فقط توی حافظه یا یک صف غیرماندگار ثبت شده قابل قبول نیست. اگه بعدش رویداد گم بشه، از نظر قرارداد مشکل سمت گیرنده‌ست.

۷.۵ تلاش دوباره و dead letter

- فاصله تلاش‌ها بعد از تلاش اول: ۱ دقیقه، ۵ دقیقه، ۳۰ دقیقه، ۲ ساعت، ۶ ساعت و ۲۴ ساعت، هر کدوم با حداکثر ۱۰% jitter و دست‌کم به اندازه `Retry-After` . بعدش هر ۲۴ ساعت یک بار تا ۷۲ ساعت از اولین تلاش؛ آخرین تلاش درست سر ۷۲ ساعته.
- بعد از هر خطای گذرا، بقیه رویدادهای محیط شما هم حدود یک دقیقه صبر می‌کنن.
- توقف (۴۰۱/۴۰۳) و dead letter برای تیم عملیات ما هشدار می‌سازن و ما با مخاطب عملیات شما تماس می‌گیریم.
- اگه رویدادی تا ۷۲ ساعت تحویل نشه، به dead letter میره. ارسال دوباره توسط تیم ما ثبت و حسابرسی میشه، همون `event_id` اصلی رو نگه می‌داره و پنجره ۷۲ ساعته تازه می‌گیره.
- خرابی گیرنده شما بازی یا شریک دیگه رو متوقف نمی‌کنه.

۷.۶ API بازیابی رویداد

```
GET /partner/v1/events?cursor=opaque_cursor&limit=100
Authorization: Bearer <partner-api-access-token>

{ "items": [{ "event_id": "evt_0193a0e1b2c37d4e8f50a1b2c3d4e5f6",
              "type": "https://api.funlands.ir/events/game.completed",
              "schema_version": 1,
              "subject": { "format": "iss_sub", "iss": "https://auth.sandbox.partner.example", "sub":
"pw_7f3c9a1e" },
              "player_seq": 42, "occurred_at": "2026-10-31T10:00:00Z",
              "game_id": "star-runner", "game_version": null, "score": 1250, "duration_ms": null,
"result": "win" }],
  "next_cursor": "opaque_next", "has_more": false,
  "generated_at": "2026-10-31T10:01:00Z" }
```

- فقط رویدادهای خود شما برمی‌گرده. توکنش از مسیر بخش ۸ گرفته میشه و scope `events:read` لازم داره.

- هر آیتم همون فیلدهای رویداد داخل SET رو داره، به‌اضافه `type` (همون URI). بار اول بدون `cursor` صدا بزنید تا از قدیمی‌ترین رویداد شروع بشه.
- `cursor` رو تفسیر نکنید و به زمان یا شناسه تبدیلش نکنید. فقط آخرین `next_cursor` رو نگه دارید و دوباره بفرستید؛ این مقدار وقتی `has_more` برابر `false` هم هست برمی‌گرده (صفحه خالی همون cursor ارسالی رو برمی‌گردونه).
- `limit` پیش‌فرض ۵۰ و حداکثر ۲۰۰ هست. cursor یا limit نامعتبر `400 PARTNER_REPORT_INVALID` می‌گیره.
- ترتیب feed نسبت به commit امنه: هر رویداد وقتی توی feed دیده میشه که همه تراکنش‌هایی که می‌تونستن رویدادی قبل از اون بذارن تموم شده باشن (معمولا کمتر از یک ثانیه). پس رویدادی که دیرتر commit شده هیچ‌وقت پشت cursor شما جا نمی‌افته.
- مدت نگهداری feed دست‌کم ۷۲ ساعت به‌اضافه زمان قطعی توافق‌شده‌ست. عدد نهایی توی قرارداد داده میاد.

۸. API گزارش

۸.۱ گرفتن توکن

توکن گزارش با `client_credentials` و روش `private_key_jwt` گرفته میشه. این کار فقط از سرور شما انجام میشه. توکن نباید به مرورگر یا اپ برسه یا لاگ بشه.

```
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&scope=reports:read
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=<signed-client-assertion>
```

```
{ "alg": "ES256", "typ": "JWT", "kid": "partner-a-sandbox-2026-11" }

{
  "iss": "<client_id>", "sub": "<client_id>",
  "aud": "https://sandbox.funlands.example/oauth/token",
  "iat": 1793440800, "exp": 1793440860,
  "jti": "assert_01J..."
}

200 OK
Cache-Control: no-store
{ "access_token": "<partner-api-jwt>", "token_type": "Bearer",
  "expires_in": 900, "scope": "reports:read" }
```

- `iss` و `sub` هر دو برابر `client_id`، `aud` دقیقا نشانی token endpoint همون محیط، عمر حداکثر ۶۰ ثانیه و `jti` یک‌بارمصرف.
- توکن launch یا logout رو به‌جای assertion نفرستید. رد میشه.
- توکن گزارش ۱۵ دقیقه عمر داره و refresh نداره. وقتی تموم شد یک assertion تازه بسازید و توکن جدید بگیرید.
- توکن گزارش یک JWT با هدر `typ` برابر `at+jwt` و `aud` برابر `partner-api` هست که فان‌لندز امضاش می‌کنه. فقط با `Authorization: Bearer` بفرستیدش و به محتواش تکیه نکنید.
- اگه از سقف نرخ توکن برای `client_id` خودتون رد بشید، جواب `429` با `"error": "temporarily_unavailable"` و هدر `Retry-After` هست. مشکل گذرا سمت ما هم `503` با همین `error` و `Retry-After` میده.
- `partner_id` فقط از همین توکن خونده میشه. هیچ پارامتری توی URL یا بدنه نمی‌تونه شریک رو عوض کنه.

۸.۲ مسیرها

مسیر	scope	محتوا
`GET /partner/v1/metrics/summary`	`reports:read`	بازه، منطقه زمانی، DAU و MAU، شروع و تکمیل بازی، زمان بازی
`GET /partner/v1/metrics/daily`	`reports:read`	ردیف روزانه با تاریخ محلی شما و بازه UTC متناظرش
`GET /partner/v1/games`	`reports:read`	شاخص‌ها به تفکیک `game_id` . فیلد `game_version` فعلا همیشه `null` هست.
`GET /partner/v1/players/{external_sub}`	`players:read`	حداقل پروفایل و آمار تجمیعی یک بازیکن خودتون در همین محیط. هر جستجو حسابرسی میشه. `age_band` فعلا همیشه `null` هست.
`GET /partner/v1/players/{external_sub}/activity`	`players:read`	فعالیت بازیکن با صفحه‌بندی cursor و فیلتر بازه (`limit` پیش‌فرض ۵۰، حداکثر ۲۰۰). `result` رویدادهای `game.completed` فعلا همیشه `null` هست.
`GET /partner/v1/events`	`events:read`	feed بازیابی رویداد (بخش ۷.۶)

خروجی فایل با `POST /partner/v1/exports` در مرحله بعد اضافه میشه.

- `sub` ناشناخته، `sub` شریک دیگه و `sub` ی که فقط توی محیط دیگه شما بازی کرده همه یک جواب یکسان می‌گیرن: `404` `PARTNER_PLAYER_NOT_FOUND`.
- `from` / `to` با قالب `YYYY-MM-DD` و تاریخ محلی، هر دو سر شامل، حداکثر ۳۶۶ روز. پارامتر نامعتبر، cursor دستکاری‌شده یا `limit` بیرون از ۱ تا ۲۰۰ جواب `400 PARTNER_REPORT_INVALID` می‌گیره.
- گزارش‌ها فقط بازیکن‌های همون محیطی رو می‌شمرن که توکن براش صادر شده.

۸.۳ تعریف شاخص‌ها

این تعریف‌ها توی پنل و API یکی هستن. هیچ‌کدوم مبنای صورتحساب نیست، مگر اینکه جدا تعریف و امضا بشه.

شاخص	تعریف
DAU	تعداد `external_sub` یکتا که در یک روز، به منطقه زمانی شما، دست‌کم یک `game.started` معتبر داشتن
MAU	همون شمارش برای ماه تقویمی‌ای که `to` توش هست، محدود به بازه درخواست: از دیرترین `from` و روز اول اون ماه تا آخر روز `to`
نرخ تکمیل	تعداد `game.completed` معتبر تقسیم بر `game.started` معتبر در همون cohort شروع
ماندگاری روز N	سهم کاربرهای cohort اولین بازی که روز N دوباره `game.started` داشتن
زمان بازی	از بازه‌های معتبر نشست و بازی، با سقف و حذف تکرار. جمع کور مدت‌هایی که کلاینت فرستاده نیست. زمانی که دستگاه گزارش می‌کنه فقط telemetry حساب میشه.

هر پاسخ گزارش این سه فیلد رو داره:

```
{
  "range": { "from": "2026-10-01", "to": "2026-10-31", "timezone": "Asia/Tehran" },
  "metrics": { "...": "..." },
  "generated_at": "2026-11-01T03:10:00Z",
  "data_lag_seconds": 2100,
  "metric_definition_version": 1
}
```

`generated_at` زمان ساخت پاسخه، `data_lag_seconds` نشون میده داده چقدر عقب‌تر از زمان واقعیه، و `metric_definition_version` نسخه تعریف‌های بالاست (عدد صحیح). اگه تعریفی عوض بشه نسخه بالا میره. `data_lag_seconds` فعلا حدود ۲۱۰۰ ثانیه‌ست، چون یک نشست بازی باز حداکثر ۳۰ دقیقه بعد بسته میشه و بستنش تا ۵ دقیقه دیگه طول می‌کشه. بازی‌های نیمه‌کاره و رویدادهای دیررس قاعده اصلاح مشخص دارن که توی قرارداد داده میاد. شکل کامل هر پاسخ توی OpenAPI هست.

۹. خطاها

همه خطاها این شکل رو دارن. تصمیم کلاینت فقط بر اساس `code` باشه. `message` متن فارسی برای نمایش به کاربره و ممکنه عوض بشه. شناسه درخواست توی هدر پاسخ `X-Correlation-Id` میاد و توی خطاهای endpointهای شریک (`/v1/partner/*`، `/v1/session/bootstrap` و `/partner/v1/*`) همون مقدار توی بدنه با نام `correlation_id` هم هست؛ وقتی با ما تماس می‌گیرید اون رو بفرستید. `retry_after_seconds` فقط توی خطاهای گذرا میاد و همون مقدار توی هدر `Retry-After` هم هست. جزئیات امضا، وجود یا نبود کاربر و داده شریک دیگه توی خطا نمیاد.

```
X-Correlation-Id: req_01J...

{ "state": false,
  "code": "TOKEN_EXPIRED",
  "message": "زمان اعتبار توکن گذشته است",
  "correlation_id": "req_01J..." }
```

کد	HTTP	معنی	کار شریک
`ORIGIN_NOT_ALLOWED`	۴۰۳	Origin صفحه ثبت نشده، خالیه یا `null`	Origin دقیق رو ثبت و تست کنید
`PARTNER_UNKNOWN`	۴۰۱	`iss` با هیچ شریکی در این محیط نمی‌خونه	issuer و محیط رو چک کنید
`PARTNER_SUSPENDED`	۴۰۳	شریک یا این محیط غیرفعاله	با پشتیبانی هماهنگ کنید، تلاش خودکار نکنید
`TOKEN_TYPE_INVALID`	۴۰۱	`typ` اشتباه یا نبودنش، یا وجود `x5u` / `jku` / `jwk` در هدر	هدر رو طبق همین راهنما بسازید
`ALG_NOT_ALLOWED`	۴۰۱	الگوریتم با مقدار ثبت‌شده یکی نیست	تنظیم امضا رو درست کنید
`KEY_NOT_FOUND`	۴۰۱	`kid` حتی بعد از خوندن دوباره JWKS پیدا نشد یا ابطال شده	کلید رو قبل از استفاده منتشر کنید
`SIGNATURE_INVALID`	۴۰۱	امضا تایید نشد	کلید و محیط رو چک کنید

`AUDIENCE_MISMATCH`	۴۰۱	`aud` اشتباهه، مثلا توکن sandbox روی production	audience همون محیط و همون نوع توکن رو بذارید
`TOKEN_EXPIRED`	۴۰۱	`exp` گذشته، عمر بیشتر از ۱۲۰ ثانیه‌ست یا `iat` در آینده‌ست	ساعت سرور رو هماهنگ کنید و توکن تازه بسازید
`TOKEN_REPLAYED`	۴۰۱	`jti` قبلا مصرف شده	برای هر ورود توکن تازه بسازید
`CLAIMS_INVALID`	۴۰۰	claim ناقص یا نامعتبر، `game` با فرم نمی‌خونه یا verifier با challenge نمی‌خونه	ساخت توکن و فرم رو اصلاح کنید
`LAUNCH_PAYLOAD_CHANGED`	۴۰۹	همون `jti` با محتوای متفاوت	برای محتوای جدید `jti` جدید بسازید
`PLAYER_SUSPENDED`	۴۰۳	بازیکن تعلیق شده	پیام مناسب نشون بدید، تلاش خودکار بی‌پایان نکنید
`BOOTSTRAP_INVALID`	۴۰۰	کد bootstrap اشتباه، مصرف‌شده، منقضی یا verifier نادرست	از مسیر relaunch ورود تازه بدید
`TOKEN_REUSE`	۴۰۱	refreshی که قبلا چرخیده دوباره استفاده شده و همه نشست‌های بازیکن باطل شدن. refresh ناشناخته یا منقضی `TOKEN_EXPIRED` می‌گیره.	ورود تازه
`SESSION_REVOKED`	۴۰۱	نشست با خروج یا غیرفعال شدن شریک یا محیط بسته شده و refresh یا bootstrap قبل از حذف نشست رسیده. بعد از حذف نشست، همون حالت `TOKEN_EXPIRED` می‌گیره (بخش ۴.۷).	مثل `TOKEN_EXPIRED` روی refresh: نشست تموم شده؛ اگه کاربر هنوز پیش شما وارده، ورود تازه
`PARTNER_TOKEN_MISMATCH`	۴۰۳	توکن بازیکن شریک بدون `x-partner-api-key` همون شریک، یا با کلید شریک دیگه، فرستاده شده	صفحه بازی کلید `client_direct` رو روی همه درخواست‌ها و اتصال‌های WebSocket بفرسته؛ تلاش خودکار نکنید
`PARTNER_KEY_INVALID`	۴۰۱	`x-partner-api-key` ناشناخته‌ست یا چرخیده	کلید فعلی برگ شریک رو بذارید؛ این خطا با refresh درست نمیشه
`SCOPE_NOT_ALLOWED`	۴۰۳	توکن API این scope رو نداره	scope درست رو درخواست کنید
`PARTNER_REPORT_INVALID`	۴۰۰	`from` ، `to` ، `cursor` یا `limit` گزارش نامعتبره	پارامترها رو طبق بخش ۸.۲ درست کنید
`PARTNER_PLAYER_NOT_FOUND`	۴۰۴	این `external_sub` توی این محیط بازیکن شما نیست	`sub` و محیط توکن رو چک کنید، تلاش خودکار نکنید
`RATE_LIMITED`	۴۲۹	از محدودیت نرخ رد شدید	بعد از `retry_after_seconds` دوباره تلاش کنید

`UPSTREAM_UNAVAILABLE`	۵۰۳	مشکل گذرا سمت ما	با رعایت `Retry-After` دوباره تلاش کنید و پیام قابل بازیابی نشون بدید

مسیر `/oauth/token` طبق استاندارد OAuth خطا میده: `invalid_request`، `invalid_client`، `unauthorized_client`، `unsupported_grant_type`، `invalid_scope` یا `temporarily_unavailable` (برای `429` و `503`، همراه هدر `Retry-After`) در فیلد `error`، به‌اضافه `error_description`.

۱۰. نمونه پیاده‌سازی سمت شریک با NestJS

این نمونه‌ها کارهای سمت سرور شریک رو با NestJS نشون میدن: انتشار JWKS، امضای توکن launch و صفحه ورود، گرفتن وب‌هوک، گرفتن توکن و صدا زدن API گزارش، و فرستادن توکن خروج. کد برای NestJS نسخه ۱۰ به بالا، Node.js نسخه ۲۰ به بالا و کتابخونه `jose` نسخه ۵ یا ۶ نوشته شده و با Express کار می‌کنه. همه نشانی‌ها، `kid` ها و شناسه‌ها نمونه‌ان و هیچ کلید واقعی‌ای اینجا نیست. اسم فایل‌ها و کلاس‌ها پیشنهادیه و می‌تونید با ساختار پروژه خودتون جورش کنید.

```
npm install @nestjs/config jose pg
npm install -D @types/pg
```

۱۰.۱ تنظیمات

همه مقدارهای هر محیط از متغیرهای محیطی خونده میشن. مقدارهای سمت فان‌لندز رو از برگ شریک بردارید و برای sandbox و production جدا نگه دارید. کلید خصوصی توی این فایل یا توی `.env` نمیاد. فقط مسیر پوشه‌ای که secret روش mount شده اینجاست.

```
// src/funlands/funlands.config.ts
import { ConfigType, registerAs } from '@nestjs/config';

function required(name: string): string {
  const value = process.env[name];
  if (!value) throw new Error(`Missing environment variable ${name}`);
  return value;
}

export const funlandsConfig = registerAs('funlands', () => ({
  // our own values
  issuer: required('PARTNER_ISSUER'),                 // https://auth.sandbox.partner.example
  signingKeyDir: required('PARTNER_SIGNING_KEY_DIR'), // mounted secret dir, one <kid>.pem per key
  activeKid: required('PARTNER_ACTIVE_KID'),          // partner-a-sandbox-2026-11
  // values Funlands gives us in the partner sheet
  funlandsBaseUrl: required('FUNLANDS_BASE_URL'),     // https://sandbox.funlands.example (JWKS, token,
reports, logout)
  funlandsGameOrigin: required('FUNLANDS_GAME_ORIGIN'), // https://play.sandbox.funlands.example (launch
form, iframe, postMessage)
  launchAudience: required('FUNLANDS_LAUNCH_AUD'),    // <base>/partner-launch
  logoutAudience: required('FUNLANDS_LOGOUT_AUD'),    // <base>/partner-logout
  tokenEndpoint: required('FUNLANDS_TOKEN_URL'),      // <base>/oauth/token
  clientId: required('FUNLANDS_CLIENT_ID'),
  funlandsJwksUrl: required('FUNLANDS_JWKS_URL'),     // <base>/.well-known/jwks.json
  setIssuer: required('FUNLANDS_SET_ISSUER'),         // https://auth.sandbox.funlands.example
  setAudience: required('FUNLANDS_SET_AUDIENCE'),     // registered for us, e.g. our issuer
}));

export type FunlandsConfig = ConfigType<typeof funlandsConfig>;
```

۱۰.۲ ساخت کلید و انتشار JWKS

این اسکریپت یک جفت کلید ES۲۵۶ می‌سازه، کلید خصوصی رو به شکل PKCS#۸ توی پوشه secret می‌نویسه و بخش عمومی رو چاپ می‌کنه. اگه فایلی با همون `kid` وجود داشته باشه، کار رو ادامه نمیده. این روش برای sandbox مناسبه. توی production کلید رو داخل KMS یا HSM بسازید و فقط کلید عمومی رو بیرون بیارید.

```
// scripts/generate-signing-key.ts
// usage: ts-node scripts/generate-signing-key.ts <kid> <secret-dir>
import { writeFile } from 'node:fs/promises';
import { join } from 'node:path';
import { exportJWK, exportPKCS8, generateKeyPair } from 'jose';

async function main(): Promise<void> {
  const [kid, secretDir] = process.argv.slice(2);
  if (!kid || !secretDir) throw new Error('usage: generate-signing-key <kid> <secret-dir>');

  const { publicKey, privateKey } = await generateKeyPair('ES256', { extractable: true });

  // flag 'wx' refuses to overwrite: a kid is never reused for a new key
  const pem = await exportPKCS8(privateKey);
  await writeFile(join(secretDir, `${kid}.pem`), pem, { mode: 0o600, flag: 'wx' });

  // print only the public part, for review before it goes into the JWKS
  const { kty, crv, x, y } = await exportJWK(publicKey);
  console.log(JSON.stringify({ kty, crv, x, y, kid, alg: 'ES256', use: 'sig' }, null, 2));
}

main().catch((err) => {
  console.error(err);
  process.exit(1);
});
```

سرویس کلید همه فایل‌های پوشه secret رو می‌خونه، چک می‌کنه کلیدها P-۲۵۶ باشن و فقط بخش عمومی رو منتشر می‌کنه. برای چرخش، فایل K۲ رو کنار K۱ بذارید تا هر دو منتشر بشن. دست‌کم ۲۴ ساعت بعد `PARTNER_ACTIVE_KID` رو به K۲ عوض کنید و فایل K۱ رو دست‌کم ۲۴ ساعت بعد از آخرین امضا بردارید (بخش ۳.۴).

```
// src/funlands/signing-key.service.ts
import { Inject, Injectable, OnModuleInit } from '@nestjs/common';
import { createPrivateKey, createPublicKey, KeyObject } from 'node:crypto';
import { readdir, readFile } from 'node:fs/promises';
import { basename, join } from 'node:path';
import { funlandsConfig, FunlandsConfig } from './funlands.config';

export interface PublicJwk {
  kty: string; crv: string; x: string; y: string;
  kid: string; alg: 'ES256'; use: 'sig';
}

@Injectable()
export class SigningKeyService implements OnModuleInit {
  private readonly privateKeys = new Map<string, KeyObject>();
  private publicKeys: PublicJwk[] = [];

  constructor(@Inject(funlandsConfig.KEY) private readonly cfg: FunlandsConfig) {}

  async onModuleInit(): Promise<void> {
    const files = (await readdir(this.cfg.signingKeyDir)).filter((f) => f.endsWith('.pem'));
    for (const file of files) {
      const kid = basename(file, '.pem');
      const key = createPrivateKey(await readFile(join(this.cfg.signingKeyDir, file), 'utf8'));
```

```
      const curve = key.asymmetricKeyDetails?.namedCurve;
      if (key.asymmetricKeyType !== 'ec' || curve !== 'prime256v1') {
        throw new Error(`Signing key ${kid} is not EC P-256`);
      }
      this.privateKeys.set(kid, key);
    }
    if (!this.privateKeys.has(this.cfg.activeKid)) {
      throw new Error(`No key file for active kid ${this.cfg.activeKid}`);
    }
    // every key in the directory is published: old and new overlap during rotation
    this.publicKeys = [...this.privateKeys].map(([kid, key]) => {
      const jwk = createPublicKey(key).export({ format: 'jwk' });
      return { kty: jwk.kty!, crv: jwk.crv!, x: jwk.x!, y: jwk.y!,
               kid, alg: 'ES256', use: 'sig' };
    });
  }

  /** The key every outgoing token is signed with. */
  active(): { kid: string; key: KeyObject } {
    return { kid: this.cfg.activeKid, key: this.privateKeys.get(this.cfg.activeKid)! };
  }

  /** Public fields only. d never leaves this service. */
  jwks(): { keys: PublicJwk[] } {
    return { keys: this.publicKeys };
  }
}
```

کنترلر JWKS فقط خروجی بالا رو با `Cache-Control` پنج‌دقیقه‌ای برمی‌گردونه. این مسیر باید دقیقا روی همون نشانی‌ای باشه که توی برگ شریک ثبت کردید و ریدایرکت نداشته باشه. اگه برای برنامه `setGlobalPrefix` گذاشتید، این مسیر رو از پیشوند مستثنا کنید.

```
// src/funlands/jwks.controller.ts
import { Controller, Get, Header } from '@nestjs/common';
import { SigningKeyService } from './signing-key.service';

@Controller('.well-known')
export class JwksController {
  constructor(private readonly keys: SigningKeyService) {}

  // GET https://auth.sandbox.partner.example/.well-known/jwks.json
  @Get('jwks.json')
  @Header('Cache-Control', 'public, max-age=300')
  getJwks() {
    return this.keys.jwks();
  }
}
```

۱۰.۳ امضای توکن launch و صفحه ورود

برای هر ورود یک verifier تازه ساخته میشه و `launch_challenge` از روی اون حساب میشه. `iat` و `exp` از یک بار خوندن ساعت میان تا فاصله‌شون دقیقا ۱۲۰ ثانیه بمونه. claimهای اختیاری فقط وقتی مقدار دارن توی توکن میان.

```
// src/funlands/launch-token.service.ts
import { Inject, Injectable } from '@nestjs/common';
import { createHash, randomBytes, randomUUID } from 'node:crypto';
import { SignJWT } from 'jose';
import { funlandsConfig, FunlandsConfig } from './funlands.config';
import { SigningKeyService } from './signing-key.service';

export interface LaunchUser {
  pairwiseSub: string; // stable pseudonymous id for Funlands only, max 128 chars
```

```
  sessionId: string;   // our session id, sent as sid
  locale?: string;     // fa-IR
  ageBand?: string;    // one of the values agreed in the partner sheet
  nickname?: string;   // max 24 chars
}

export interface LaunchForm {
  launchToken: string;
  bootstrapVerifier: string;
  game: string;
}

@Injectable()
export class LaunchTokenService {
  constructor(
    @Inject(funlandsConfig.KEY) private readonly cfg: FunlandsConfig,
    private readonly keys: SigningKeyService,
  ) {}

  async create(user: LaunchUser, game: string): Promise<LaunchForm> {
    const bootstrapVerifier = randomBytes(32).toString('base64url');
    const launchChallenge = createHash('sha256')
      .update(bootstrapVerifier, 'ascii')
      .digest('base64url');
    const { kid, key } = this.keys.active();
    const now = Math.floor(Date.now() / 1000);

    const launchToken = await new SignJWT({
      sid: user.sessionId,
      game,
      launch_challenge: launchChallenge,
      ...(user.locale ? { locale: user.locale } : {}),
      ...(user.ageBand ? { age_band: user.ageBand } : {}),
      ...(user.nickname ? { nickname: user.nickname } : {}),
    })
      .setProtectedHeader({ alg: 'ES256', typ: 'funlands-launch+jwt', kid })
      .setIssuer(this.cfg.issuer)
      .setAudience(this.cfg.launchAudience)
      .setSubject(user.pairwiseSub)
      .setIssuedAt(now)
      .setExpirationTime(now + 120)
      .setJti(randomUUID())
      .sign(key);

    return { launchToken, bootstrapVerifier, game };
  }
}
```

کنترلر صفحه بازی اول چک می‌کنه کاربر وارد شده باشه و این بازی براش مجاز باشه. بعد یک صفحه HTML با `no-store` و CSP سخت برمی‌گردونه که `frame-src` و `form-action` فقط Origin بازی فان‌لندز (`FUNLANDS_GAME_ORIGIN`) رو مجاز می‌کنه. `Referrer-Policy` عمدا `strict-origin-when-cross-origin` هست تا مرورگر Origin واقعی صفحه رو بفرسته (هشدار بخش ۴ رو ببینید). مسیر دوم برای relaunch توکن تازه میده.

```
// src/funlands/play.controller.ts
import {
  Controller, Get, Header, Inject, NotFoundException, Param, Post, Req, Res, UseGuards,
} from '@nestjs/common';
import type { Request, Response } from 'express';
import { randomBytes } from 'node:crypto';
import { funlandsConfig, FunlandsConfig } from './funlands.config';
import { LaunchForm, LaunchTokenService, LaunchUser } from './launch-token.service';
import { renderPlayPage } from './play-page';
```

```
import { GameAccessService } from '../games/game-access.service';    // your entitlement check
import { PartnerSessionGuard } from '../auth/partner-session.guard'; // your login guard

interface AuthedRequest extends Request { user: LaunchUser }
const GAME_ID = /^[a-z0-9-]{1,64}$/;

@Controller('games')
@UseGuards(PartnerSessionGuard)
export class PlayController {
  constructor(
    @Inject(funlandsConfig.KEY) private readonly cfg: FunlandsConfig,
    private readonly launches: LaunchTokenService,
    private readonly access: GameAccessService,
  ) {}

  @Get(':game/play')
  async play(
    @Param('game') game: string,
    @Req() req: AuthedRequest,
    @Res() res: Response,
  ): Promise<void> {
    await this.assertAllowed(req.user, game);
    const form = await this.launches.create(req.user, game);
    const origin = new URL(this.cfg.funlandsGameOrigin).origin; // launch and bootstrap run here, not on the
API base
    const nonce = randomBytes(16).toString('base64');

    res.status(200).set({
      'Content-Type': 'text/html; charset=utf-8',
      'Cache-Control': 'no-store',
      'Referrer-Policy': 'strict-origin-when-cross-origin', // keeps the real Origin on the cross-origin form
POST
      'Content-Security-Policy': [
        "default-src 'none'", `script-src 'nonce-${nonce}'`, `style-src 'nonce-${nonce}'`,
        `frame-src ${origin}`, `form-action ${origin}`, "connect-src 'self'",
        "frame-ancestors 'self'", "base-uri 'none'",
      ].join('; '),
    });
    res.send(renderPlayPage({ form, launchUrl: `${origin}/v1/partner/launch`,
      gameOrigin: origin, relaunchPath: `/games/${game}/launch`, nonce }));
  }

  /** Fresh launch for funlands:relaunch_requested. Protect it with your usual CSRF defence. */
  @Post(':game/launch')
  @Header('Cache-Control', 'no-store')
  async relaunch(@Param('game') game: string, @Req() req: AuthedRequest): Promise<LaunchForm> {
    await this.assertAllowed(req.user, game);
    return this.launches.create(req.user, game);
  }

  private async assertAllowed(user: LaunchUser, game: string): Promise<void> {
    if (!GAME_ID.test(game) || !(await this.access.canPlay(user, game))) {
      throw new NotFoundException();
    }
  }
}
```

خود صفحه یک iframe و یک فرم مخفی داره که بلافاصله `launch_token`، `game` و `bootstrap_verifier` رو با POST داخل iframe می‌فرسته. همه مقدارها escape میشن. اگه صفحه بازی پیام `funlands:relaunch_requested` بفرسته، فقط وقتی Origin بازی و منبع

پیام درست باشن توکن تازه گرفته میشه، جواب `partner:relaunch_ready` با همون `request_id` برمی‌گرده و فرم دوباره ارسال میشه. `funlands:launch_failed` از صفحه handoff فقط لاگ میشه (بخش ۴.۹).

```
// src/funlands/play-page.ts
import { LaunchForm } from './launch-token.service';

const ESCAPES: Record<string, string> = {
  '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;',
};
const esc = (value: string): string => value.replace(/[&<>"']/g, (c) => ESCAPES[c]);

export interface PlayPageInput {
  form: LaunchForm;
  launchUrl: string;    // https://play.sandbox.funlands.example/v1/partner/launch
  gameOrigin: string;   // https://play.sandbox.funlands.example (FUNLANDS_GAME_ORIGIN)
  relaunchPath: string; // /games/star-runner/launch
  nonce: string;
}

export function renderPlayPage(input: PlayPageInput): string {
  const { form, launchUrl, gameOrigin, relaunchPath, nonce } = input;
  return `<!doctype html>
<html lang="fa" dir="rtl">
<head>
  <meta charset="utf-8">
  <title>Game</title>
  <style nonce="${nonce}">
    html, body, iframe { margin: 0; width: 100%; height: 100%; border: 0; }
  </style>
</head>
<body>
  <iframe name="game-frame" title="Game" allow="fullscreen"></iframe>
  <form id="launch" method="post" action="${esc(launchUrl)}" target="game-frame">
    <input type="hidden" name="launch_token" value="${esc(form.launchToken)}">
    <input type="hidden" name="game" value="${esc(form.game)}">
    <input type="hidden" name="bootstrap_verifier" value="${esc(form.bootstrapVerifier)}">
  </form>
  <script nonce="${nonce}">
    const GAME_ORIGIN = ${JSON.stringify(gameOrigin)};
    const RELAUNCH = ${JSON.stringify(relaunchPath)};
    const form = document.getElementById('launch');
    const frame = document.querySelector('iframe');
    form.submit();

    window.addEventListener('message', async (event) => {
      if (event.origin !== GAME_ORIGIN || event.source !== frame.contentWindow) return;
      if (event.data && event.data.type === 'funlands:launch_failed') {
        console.warn('Funlands launch failed', event.data.code, event.data.request_id); // request_id = X-
Correlation-Id
        return;
      }
      // Sent by the game page itself when it has no session (reload, or TOKEN_EXPIRED / SESSION_REVOKED on
refresh).
      if (!event.data || event.data.type !== 'funlands:relaunch_requested') return;
      const res = await fetch(RELAUNCH, { method: 'POST', credentials: 'same-origin' });
      if (!res.ok) return; // show a retry message here
      const next = await res.json();
      frame.contentWindow.postMessage(
        { type: 'partner:relaunch_ready', request_id: event.data.request_id }, GAME_ORIGIN);
      form.elements.launch_token.value = next.launchToken;
      form.elements.bootstrap_verifier.value = next.bootstrapVerifier;
      form.elements.game.value = next.game;
```

```
      form.submit();
    });
  </script>
</body>
</html>`;
}
```

۱۰.۴ گرفتن و چک کردن وب‌هوک

بدنه وب‌هوک یک JWT خامه، نه JSON. پس توی `main.ts` یک parser متنی با سقف حجم برای نوع `application/secevent+jwt` اضافه کنید.

```
// src/main.ts
import { NestFactory } from '@nestjs/core';
import { NestExpressApplication } from '@nestjs/platform-express';
import { AppModule } from './app.module';

async function bootstrap(): Promise<void> {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  // webhook bodies arrive as a raw JWT string, capped in size
  app.useBodyParser('text', { type: 'application/secevent+jwt', limit: '64kb' });
  await app.listen(3000);
}

void bootstrap();
```

سرویس چک کردن SET کلید رو فقط از نشانی JWKS فان‌لندز که توی برگ شریک گرفتید می‌خونه و `createRemoteJWKSet` کش و دوباره خوندن برای `kid` ناشناخته رو مدیریت می‌کنه. هدرهای ممنوع و `kid` های ابطال‌شده قبل از چک امضا رد میشن.

```
// src/funlands/set-verifier.service.ts
import {
  BadRequestException, Inject, Injectable, ServiceUnavailableException, UnauthorizedException,
} from '@nestjs/common';
import {
  createRemoteJWKSet, decodeProtectedHeader, errors, jwtVerify,
  JWTPayload, ProtectedHeaderParameters,
} from 'jose';
import { funlandsConfig, FunlandsConfig } from './funlands.config';
import { KidDenylist } from './kid-denylist';

export interface FunlandsEvent {
  type: string;          // https://api.funlands.ir/events/game.completed
  event_id: string;
  schema_version: number;
  subject: { format: 'iss_sub'; iss: string; sub: string };
  player_seq: number;
  occurred_at: string;
  [field: string]: unknown;
}

const FORBIDDEN_HEADERS = ['jku', 'x5u', 'jwk', 'x5c'];

@Injectable()
export class SetVerifierService {
  private readonly jwks: ReturnType<typeof createRemoteJWKSet>;

  constructor(
    @Inject(funlandsConfig.KEY) private readonly cfg: FunlandsConfig,
    private readonly denylist: KidDenylist,
  ) {
```

```
    this.jwks = createRemoteJWKSet(new URL(cfg.funlandsJwksUrl), {
      timeoutDuration: 3_000,   // JWKS fetch timeout
      cooldownDuration: 30_000, // unknown kid: at most one refetch every 30 s
      cacheMaxAge: 600_000,     // never keep Funlands keys longer than 10 min
    });
  }
```

بعد امضا، `typ` ، `iss` ، `aud` و سن `iat` چک میشن. خطای امضا و claim جواب `401` می‌گیره و ما ارسال رو نگه می‌داریم. timeout خوندن JWKS جواب `503` می‌گیره تا دوباره تلاش کنیم. رویدادی که شکلش درست نیست `400` می‌گیره.

```
  async verify(token: string): Promise<FunlandsEvent[]> {
    let header: ProtectedHeaderParameters;
    try {
      header = decodeProtectedHeader(token);
    } catch {
      throw new UnauthorizedException();
    }
    const forbidden = FORBIDDEN_HEADERS.some((h) => h in header);
    if (forbidden || !header.kid || this.denylist.has(header.kid)) {
      throw new UnauthorizedException();
    }

    let payload: JWTPayload;
    try {
      ({ payload } = await jwtVerify(token, this.jwks, {
        algorithms: ['ES256'],
        typ: 'secevent+jwt',
        issuer: this.cfg.setIssuer,
        audience: this.cfg.setAudience,
        requiredClaims: ['iat', 'jti', 'events'],
        maxTokenAge: '5m',  // iat at most 5 min old
        clockTolerance: 30, // and at most 30 s in the future
      }));
    } catch (err) {
      // JWKS timeout: 503 so Funlands retries. Bad signature or claims: 401.
      if (err instanceof errors.JWKSTimeout) throw new ServiceUnavailableException();
      if (err instanceof errors.JOSEError) throw new UnauthorizedException();
      throw err;
    }

    const events = payload.events;
    if (!events || typeof events !== 'object' || Object.keys(events).length === 0) {
      throw new BadRequestException();
    }
    const entries = Object.entries(events as Record<string, Partial<FunlandsEvent>>);
    return entries.map(([type, body]) => {
      if (typeof body?.event_id !== 'string' || typeof body.player_seq !== 'number') {
        throw new BadRequestException();
      }
      return { ...body, type } as FunlandsEvent;
    });
  }
}
```

denylist باید جایی باشه که وسط حادثه بشه بدون deploy عوضش کرد. این نمونه برای کوتاهی از متغیر محیطی می‌خونه.

```
// src/funlands/kid-denylist.ts
import { Injectable } from '@nestjs/common';

/** Leaked Funlands kids. In production, load from a store you can change without a deploy. */
@Injectable()
export class KidDenylist {
```

```
  private readonly kids = new Set(
    (process.env.FUNLANDS_KID_DENYLIST ?? '').split(',').filter(Boolean),
  );

  has(kid: string): boolean {
    return this.kids.has(kid);
  }
}
```

تشخیص تکرار روی `event_id` انجام میشه، با کلید اصلی جدول. رویداد تکراری بی‌صدا نادیده گرفته میشه و باز هم جواب موفق می‌گیره.

```
CREATE TABLE funlands_event_inbox (
  event_id     text PRIMARY KEY,
  event_type   text        NOT NULL,
  player_sub   text        NOT NULL,
  player_seq   bigint      NOT NULL,
  occurred_at  timestamptz NOT NULL,
  payload      jsonb       NOT NULL,
  received_at  timestamptz NOT NULL DEFAULT now(),
  processed_at timestamptz
);
```

```
// src/funlands/event-inbox.ts
import { Inject, Injectable } from '@nestjs/common';
import { Pool } from 'pg';
import { FunlandsEvent } from './set-verifier.service';

export const PG_POOL = Symbol('PG_POOL');

export abstract class EventInbox {
  /** Resolves only after the events are committed. Duplicates are ignored, not rejected. */
  abstract saveAll(events: FunlandsEvent[]): Promise<void>;
}

@Injectable()
export class PgEventInbox extends EventInbox {
  constructor(@Inject(PG_POOL) private readonly pool: Pool) {
    super();
  }

  async saveAll(events: FunlandsEvent[]): Promise<void> {
    const client = await this.pool.connect();
    try {
      await client.query('BEGIN');
      for (const e of events) {
        await client.query(
          `INSERT INTO funlands_event_inbox
             (event_id, event_type, player_sub, player_seq, occurred_at, payload)
           VALUES ($1, $2, $3, $4, $5, $6)
           ON CONFLICT (event_id) DO NOTHING`,
          [e.event_id, e.type, e.subject.sub, e.player_seq, e.occurred_at, JSON.stringify(e)],
        );
      }
      await client.query('COMMIT');
    } catch (err) {
      await client.query('ROLLBACK');
      throw err;
    } finally {
      client.release();
    }
  }
}
```

کنترلر فقط وقتی `202` برمی‌گردونه که رویدادها commit شده باشن. اگه ذخیره شکست بخوره، Nest خودش `500` برمی‌گردونه و ما دوباره می‌فرستیم. پردازش واقعی رو یک worker جدا از روی همین جدول انجام بده. اونجا آخرین `player_seq` هر بازیکن رو نگه دارید و اگه فاصله دیدید از feed بازیابی پرش کنید.

```
// src/funlands/funlands-webhook.controller.ts
import { BadRequestException, Body, Controller, Headers, HttpCode, Post } from '@nestjs/common';
import { EventInbox } from './event-inbox';
import { SetVerifierService } from './set-verifier.service';

@Controller('funlands')
export class FunlandsWebhookController {
  constructor(
    private readonly verifier: SetVerifierService,
    private readonly inbox: EventInbox,
  ) {}

  // POST https://sandbox.partner.example/funlands/events
  @Post('events')
  @HttpCode(202)
  async receive(
    @Headers('content-type') contentType: string | undefined,
    @Body() body: unknown,
  ): Promise<void> {
    if (!contentType?.startsWith('application/secevent+jwt') || typeof body !== 'string') {
      throw new BadRequestException();
    }
    const events = await this.verifier.verify(body.trim()); // 401, 400 or 503 on failure
    await this.inbox.saveAll(events); // a DB error becomes 500, so Funlands retries
    // 202 only after commit; processing happens later in a worker that reads the inbox
  }
}
```

۱۰.۵ توکن و API گزارش

خطاهای فان‌لندز رو با یک کلاس خطا نگه دارید تا `code` و شناسه درخواست (هدر `X-Correlation-Id`) همیشه در دسترس باشن. مسیر `/oauth/token` به‌جای `code` فیلد `error` برمی‌گردونه.

```
// src/funlands/funlands-api.error.ts
export class FunlandsApiError extends Error {
  constructor(
    readonly status: number,
    readonly code: string,
    readonly correlationId?: string,
    readonly retryAfterSeconds?: number,
  ) {
    super(`Funlands ${status} ${code} correlation_id=${correlationId ?? '-'}`);
  }

  static async from(res: Response): Promise<FunlandsApiError> {
    const body = (await res.json().catch(() => ({}))) as {
      code?: string; error?: string; retry_after_seconds?: number;
    };
    const retryAfter = Number(res.headers.get('retry-after') ?? body.retry_after_seconds);
    // API errors carry code, /oauth/token errors carry the OAuth error field
    return new FunlandsApiError(res.status, body.code ?? body.error ?? 'UNKNOWN',
      res.headers.get('x-correlation-id') ?? undefined, Number.isFinite(retryAfter) ? retryAfter :
undefined);
  }
}
```

کلاینت گزارش توکن رو فقط توی حافظه نگه می‌داره و یک دقیقه قبل از تموم شدن عوضش می‌کنه. اگه API جواب `401` بده، یک بار توکن تازه می‌گیره و دوباره امتحان می‌کنه. اینجا مسیر `metrics/daily` نمونه‌ست و بقیه مسیرهای بخش ۸.۲ هم همین‌طور صدا زده میشن.

```
// src/funlands/funlands-api.client.ts
import { Inject, Injectable } from '@nestjs/common';
import { randomUUID } from 'node:crypto';
import { SignJWT } from 'jose';
import { FunlandsApiError } from './funlands-api.error';
import { funlandsConfig, FunlandsConfig } from './funlands.config';
import { SigningKeyService } from './signing-key.service';

export interface DailyMetricsResponse {
  range: { from: string; to: string; timezone: string };
  metrics: Record<string, unknown>; // exact row shape: see the OpenAPI document
  generated_at: string;
  data_lag_seconds: number;
  metric_definition_version: number;
}

@Injectable()
export class FunlandsApiClient {
  private token?: { value: string; expiresAt: number }; // memory only, never logged

  constructor(
    @Inject(funlandsConfig.KEY) private readonly cfg: FunlandsConfig,
    private readonly keys: SigningKeyService,
  ) {}

  async dailyMetrics(from: string, to: string): Promise<DailyMetricsResponse> {
    const url = new URL('/partner/v1/metrics/daily', this.cfg.funlandsBaseUrl);
    url.search = new URLSearchParams({ from, to }).toString(); // parameter names: see OpenAPI
    return this.get<DailyMetricsResponse>(url);
  }

  private async get<T>(url: URL, retried = false): Promise<T> {
    const res = await fetch(url, {
      headers: {
        Authorization: `Bearer ${await this.accessToken()}`,
        Accept: 'application/json',
      },
      signal: AbortSignal.timeout(10_000),
    });
    if (res.status === 401 && !retried) {
      this.token = undefined; // token expired or revoked: get a new one once
      return this.get<T>(url, true);
    }
    if (!res.ok) throw await FunlandsApiError.from(res);
    return (await res.json()) as T;
  }
```

برای گرفتن توکن یک client assertion با عمر ۶۰ ثانیه و `jti` تازه امضا میشه. `iss` و `sub` هر دو `client_id` هستن و `aud` دقیقا نشانی token endpoint همون محیطه.

```
  private async accessToken(): Promise<string> {
    if (this.token && this.token.expiresAt > Date.now() + 60_000) return this.token.value;

    const { kid, key } = this.keys.active();
    const now = Math.floor(Date.now() / 1000);
    const assertion = await new SignJWT({})
      .setProtectedHeader({ alg: 'ES256', typ: 'JWT', kid })
      .setIssuer(this.cfg.clientId)
      .setSubject(this.cfg.clientId)
```

```
      .setAudience(this.cfg.tokenEndpoint)
      .setIssuedAt(now)
      .setExpirationTime(now + 60)
      .setJti(randomUUID())
      .sign(key);

    const res = await fetch(this.cfg.tokenEndpoint, {
      method: 'POST',
      headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
      body: new URLSearchParams({
        grant_type: 'client_credentials',
        scope: 'reports:read',
        client_assertion_type: 'urn:ietf:params:oauth:client-assertion-type:jwt-bearer',
        client_assertion: assertion,
      }),
      signal: AbortSignal.timeout(5_000),
    });
    if (!res.ok) throw await FunlandsApiError.from(res);

    const body = (await res.json()) as { access_token: string; expires_in: number };
    this.token = { value: body.access_token, expiresAt: Date.now() + body.expires_in * 1000 };
    return body.access_token;
  }
}
```

۱۰.۶ فرستادن توکن خروج

وقتی کاربر از سرویس شما خارج میشه یا نشستش رو باطل می‌کنید، این سرویس رو صدا بزنید. اگه شبکه قطع شد یا `5xx` و `429` گرفتید، همون توکن دوباره فرستاده میشه، چون ارسال دوباره همون `jti` هم `200` می‌گیره. اگه بعد از سه بار تایید نگرفتید، خروج رو توی صف خودتون بذارید و بعدا با توکن تازه بفرستید، چون عمر هر توکن خروج ۱۲۰ ثانیه‌ست.

```
// src/funlands/funlands-logout.service.ts
import { Inject, Injectable, Logger } from '@nestjs/common';
import { randomUUID } from 'node:crypto';
import { setTimeout as sleep } from 'node:timers/promises';
import { SignJWT } from 'jose';
import { FunlandsApiError } from './funlands-api.error';
import { funlandsConfig, FunlandsConfig } from './funlands.config';
import { SigningKeyService } from './signing-key.service';

const LOGOUT_EVENT = 'http://schemas.openid.net/event/backchannel-logout';

@Injectable()
export class FunlandsLogoutService {
  private readonly logger = new Logger(FunlandsLogoutService.name);

  constructor(
    @Inject(funlandsConfig.KEY) private readonly cfg: FunlandsConfig,
    private readonly keys: SigningKeyService,
  ) {}

  /** Call when the user signs out or we revoke the session sent as sid. */
  async logout(pairwiseSub: string, sessionId: string): Promise<void> {
    const { kid, key } = this.keys.active();
    const now = Math.floor(Date.now() / 1000);
    const logoutToken = await new SignJWT({ sid: sessionId, events: { [LOGOUT_EVENT]: {} } })
      .setProtectedHeader({ alg: 'ES256', typ: 'logout+jwt', kid })
      .setIssuer(this.cfg.issuer)
      .setAudience(this.cfg.logoutAudience)
      .setSubject(pairwiseSub)
      .setIssuedAt(now)
      .setExpirationTime(now + 120) // no nonce claim
```

```
      .setJti(randomUUID())
      .sign(key);

    const url = new URL('/partner/v1/backchannel-logout', this.cfg.funlandsBaseUrl);
    for (let attempt = 1; attempt <= 3; attempt++) {
      try {
        const res = await fetch(url, {
          method: 'POST',
          headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
          body: new URLSearchParams({ logout_token: logoutToken }),
          signal: AbortSignal.timeout(5_000),
        });
        if (res.ok) return; // 200: done now or already done before
        const error = await FunlandsApiError.from(res);
        // other 4xx is a configuration problem: do not retry
        if (res.status < 500 && res.status !== 429) throw error;
        this.logger.warn(`logout attempt ${attempt} failed: ${error.message}`);
      } catch (err) {
        if (err instanceof FunlandsApiError) throw err;
        this.logger.warn(`logout attempt ${attempt} failed: network`); // never log the token
      }
      if (attempt < 3) await sleep(attempt * 2_000);
    }
    throw new Error('Funlands logout not confirmed: queue it and retry later with a fresh token');
  }
}
```

۱۰.۷ وصل کردن ماژول

همه این‌ها توی یک ماژول جمع میشن. `GameAccessService` و `PartnerSessionGuard` مال خود شماست و فقط باید بگن کاربر کیه و اجازه این بازی رو داره یا نه.

```
// src/funlands/funlands.module.ts
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { Pool } from 'pg';
import { EventInbox, PG_POOL, PgEventInbox } from './event-inbox';
import { FunlandsApiClient } from './funlands-api.client';
import { FunlandsLogoutService } from './funlands-logout.service';
import { FunlandsWebhookController } from './funlands-webhook.controller';
import { funlandsConfig } from './funlands.config';
import { JwksController } from './jwks.controller';
import { KidDenylist } from './kid-denylist';
import { LaunchTokenService } from './launch-token.service';
import { PlayController } from './play.controller';
import { SetVerifierService } from './set-verifier.service';
import { SigningKeyService } from './signing-key.service';
import { GamesModule } from '../games/games.module'; // exports your GameAccessService

@Module({
  imports: [ConfigModule.forFeature(funlandsConfig), GamesModule],
  controllers: [JwksController, PlayController, FunlandsWebhookController],
  providers: [
    SigningKeyService, LaunchTokenService, SetVerifierService, KidDenylist,
    FunlandsApiClient, FunlandsLogoutService,
    {
      provide: PG_POOL,
      useFactory: () => new Pool({ connectionString: process.env.DATABASE_URL }),
```

```
    },
    { provide: EventInbox, useClass: PgEventInbox },
  ],
  exports: [FunlandsLogoutService, FunlandsApiClient],
})
export class FunlandsModule {}
```

۱۱. چک‌لیست قبل از production

همه این موارد باید روی sandbox واقعی، نه روی نمونه‌های این سند، پاس بشن و نتیجه‌شون ثبت بشه.

کلید

- JWKS فقط کلید عمومی داره، هر کلید `kid` و `alg` داره و نشانیش بدون ریدایرکت جواب میده.
- چرخش کامل تمرین شده: انتشار K۲، امضا با K۲، قبول K۱ و K۲ در هم‌پوشانی، و حذف K۱.
- ابطال اضطراری تمرین شده: بعد از اعلام، توکن K۱ با `exp` معتبر رد میشه.
- توکن با `kid` ناشناخته، `alg=none` ، `HS256` یا هدر `jku` رد میشه.
- کلید و توکن sandbox روی production قبول نمیشه، و دو شریک با `kid` یکسان توکن هم رو باز نمی‌کنن.

ورود و نشست

- ورود معتبر، و ورود دوباره با همون `sub` که به همون بازیکن می‌رسه.
- همون `sub` توی production بعد از بازی در sandbox یک بازیکن جدا می‌سازه و گزارش هر محیط فقط بازیکن همون محیط رو نشون میده.
- دو ورود هم‌زمان یک بازیکن می‌سازن.
- توکن منقضی، عمر بیشتر از ۱۲۰ ثانیه، `iat` آینده، `aud` اشتباه و نبود `typ` رد میشن.
- Origin غایب یا ثبت‌نشده رد میشه.
- فرم launch، `frame-src` ، `form-action` و چک `postMessage` همه با Origin بازی (`FUNLANDS_GAME_ORIGIN`) همون محیط تنظیم شدن، نه با نشانی پایه API.
- برای `client_direct` ، صفحه بازی `x-partner-api-key` رو روی همه درخواست‌ها و اتصال‌های WebSocket می‌فرسته و هیچ سکرت `server_to_server` ی به مرورگر نمی‌رسه.
- verifier اشتباه رد میشه، و تکرار ورود بعد از مصرف کد طبق بخش ۴.۸ رفتار می‌کنه.
- هیچ توکن، کد یا verifierی توی URL یا لاگ‌های شما نیست.
- relaunch بعد از reload و بعد از قطع موقت سرور شما کار می‌کنه.
- کاربر با رده سنی ناشناخته محتوای محدودشده نمی‌بینه.

خروج

- بعد از توکن خروج، نشست و WebSocket فورا بسته میشن و refresh با `401 TOKEN_EXPIRED` (یا اگه زودتر از حذف نشست برسه، `SESSION_REVOKED`) رد میشه و صفحه بازی ورود تازه می‌خواد.
- توکن خروج تکراری `200` می‌گیره و توکن خروج بدون `exp` رد میشه.

رویداد

- گیرنده شما امضا، `iss` ، `aud` و سن `iat` رو چک می‌کنه و تکرار رو با `event_id` تشخیص میده.
- `202` فقط بعد از ذخیره ماندگار برمی‌گرده.
- ارسال تکراری با SET تازه و همون `event_id` فقط یک بار پردازش میشه.
- رفتار درست در ۴۲۹ با `Retry-After` ، dead letter، timeout، 5xx و replay.

- رویداد خارج از ترتیب و فاصله در `player_seq` با feed بازیابی پر میشه.

گزارش و داده

- توکن بازی برای API گزارش رد میشه و assertion تکراری رد میشه.
- با `external_sub` شریک دیگه هیچ داده‌ای برنمی‌گرده.
- عددهای پنل و API برای یک بازه یکی هستن.
- پیام‌های فارسی خطا به‌صورت دستی در حالت RTL مرور شدن.

عملیات

- کانال حادثه دوطرفه، مخاطب عملیات و مخاطب امنیت ثبت و یک بار امتحان شدن.
- عددهای کش، اختلاف ساعت و مهلت اعلام حادثه توی برگ عملیاتی دو طرف نوشته شدن.

۱۲. موارد باز

این‌ها هنوز نهایی نیستن. تا وقتی نهایی نشدن، روی هیچ شکل پیشنهادی‌شون پیاده‌سازی نکنید.

موضوع	وضعیت
API رویدادهای فنی بازی	پنل پوسته‌ها به رویدادهایی مثل Start، Run، Load، پایان مرحله و Crash نیاز داره، ولی API دریافتش هنوز تعریف نشده. یک طرح پیشنهادی داریم که هنوز تایید نشده. این عددها فقط telemetry هستن و مبنای امتیاز یا شاخص رسمی نمیشن.
ورود اپ بومی	مسیر handoff سرور به سرور، شکل درخواست و روش گره زدن به دستگاه برای iOS، Android و TV هنوز تعریف نشده (بخش ۵).
حذف حساب	نشانی درخواست، شکل پیام (مثلا یک SET امضاشده)، مسیر پیگیری وضعیت و مهلت کامل شدن حذف تا امضای سیاست داده باز می‌مونن.
پروفایل خروج	الان `typ: logout+jwt` رو قبول می‌کنیم. اینکه مقدار دیگه‌ای هم پذیرفته بشه، استفاده از پروفایل کامل OIDC Back-Channel Logout و خروج بدون `sid` هنوز توافق نشده.
فضای شریک روی درخواست‌های بازی	الان صفحه بازی شریک‌های `client_direct` باید `x-partner-api-key` رو بفرسته (بخش ۴.۷). اینکه فضای شریک از خود توکن بازیکن شریک خونده بشه در حال بررسیه و تصمیمی براش گرفته نشده. برای شریک‌های `server_to_server` هم که سکرتشون نباید به مرورگر برسه، مسیر صفحه بازی هنوز تعریف نشده.
عددهای هر شریک	مجموعه مقدارهای `age_band`، محدودیت‌های نرخ، مدت نگهداری داده، مسیر ماندگاری و خروجی فایل توی برگ هر شریک نهایی میشن.

برای سوال یا گزارش ناهماهنگی بین این راهنما و رفتار واقعی sandbox، با تیم بک‌اند فان‌لندز تماس بگیرید و `correlation_id` درخواست رو همراهش بفرستید.